

Project / Dissertation Statement of Impact [Dubai Campus Disruption 2026]

As a result of the conflict in the Middle East, teaching and learning activities for students at Heriot-Watt University Dubai campus were disrupted. The time available for completion of Final Year Projects / Dissertations was curtailed. In addition, your work may have been impacted in other ways.

The university is committed to ensuring no student is academically disadvantaged by the conflict. You are invited to outline here how the disruption impacted the development of your project / dissertation. This statement will provide context for markers so that your work can be assessed fairly, with due appreciation of context.

Student Name:	Mohamad Ajaz Imran
Student ID Number:	H00458396
Programme:	Computer Science Software Engineering
Dissertation / Project Title:	Accelerating Requirements Analysis: Machine Learning & LLMs for Automated Classification and Quality Checks to Improve Software Specification
Supervisor:	Dr. Ullah Abrar

Statement of Impact of the Disruption on Your Project / Dissertation

1. What specific impact did the disruption due to the conflict and Dubai Campus closure have on your dissertation / project? Outline the specific disruption and the sections of your work impacted.

For example:

- I was unable to access the labs, so the experiments outlined in Chapter X could not be completed.
- I was only able to complete 10 of a planned 25 interviews, so my findings and analysis are based on a smaller sample than intended.
- I was unable to complete all the planned reading due to disrupted work for X days.

Please be factual and concise. (maximum 100 words)

I was unable to conduct planned advisor meetings with my professor, preventing me from getting valuable dissertation feedback

Project / Dissertation Statement of Impact [Dubai Campus Disruption 2026]

2. What adjustments have you made to your Project / Dissertation as a result of this curtailment?

For example:

- I focused on extending the literature review and developing a detailed theoretical framework.
- I provided an extended description of the intended methodology and experimental design.
- I analysed the partial data I had collected before the disruption.
- I adapted data collection from face-to-face interviews to online.
- I curtailed my literature review and focused on xxx aspects of the topic.

Please be factual and concise. (*maximum 100 words*)

I focused on example literature to improve my dissertation implementation

Student Signature (*type your name*): Mohamad Ajaz

Date: 07/04/2026

Copy this page and insert it into your Dissertation / Project Report in front of your title page.

Accelerating Requirements Analysis: Machine Learning & LLMs for Automated Classification and Quality Checks to Improve Software Specification

by Mohamad Ajaz IMRAN

Honours Dissertation

Submitted in partial fulfilment for the degree of
B.Sc. (Hons) Software Engineering

Supervised by Prof. Ullah ABRAR



HERIOT-WATT UNIVERSITY
School of Mathematical and Computer Sciences

9th April 2026

The copyright in this dissertation is owned by the author. Any quotation from the dissertation or use of any of the information contained in it must acknowledge it as the source of the quotation or information.

AUTHORSHIP DECLARATION

Course code and name	F20PA– Honours Project
Type of assessment	Individual
Coursework title	Accelerating Requirements Analysis: Machine Learning & LLMs for Automated Classification and Quality Checks to Improve Software Specification
Student name	Mohamad Ajaz IMRAN
Student ID	H00458396

By signing this form:

- I declare that the work I have submitted for individual assessment or the work I have contributed to a group assessment is entirely my own. I have not taken the ideas, writings or inventions of another person and used them as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements sections.
- I confirm that I have read, understood and followed the University's Regulations on academic misconduct as published on the University's website, and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on Academic Integrity and Plagiarism.

Student signature: Mohamad Ajaz IMRAN

Date: 9th April 2026

Your work will not be marked if a signed copy of this form is not included with your submission.

Acknowledgments

What has been 4 years of painstaking university has turned into a lifetime of memories, and I owe an immense amount of gratitude to the people that have supported me throughout this journey.

First and foremost, I would like to thank my parents for their unwavering support and encouragement. To my mother, thank you for always being my guiding light and for your endless love. To my father, thank you for being the backbone to my success and for being an invaluable role model.

And a special thanks to Mr. Hanif Yusuf, who I wouldn't have been able to attend university without his sponsorship and support throughout these years.

To my most amazing friends, thank you for the laughter, late-night study sessions, and for being there through thick and thin. May our graduation not be the end but the beginning of lifelong friendships and adventures.

And finally, I would like to express my deepest gratitude to my supervisor, Prof. Ullah Abrar, for his guidance, patience, and insightful feedback that has been instrumental in the completion of this dissertation.

May this dissertation stand as a testament, to all those who have supported me and to the future that lies ahead.

ABSTRACT

The requirements engineering process plays a crucial role in software development and can define the success or failure of software projects, yet traditional approaches have risen to several issues such as ambiguity, inconsistency, and inefficiency. This dissertation presents PURR (Pipeline-for-Understanding-Requirements-Reliability) a hybrid pipeline that combines classical machine learning with controlled LLM prompting to automate requirements analysis and reduce ambiguity. We evaluate the system on a curated corpus of 4,000 requirements and report task-level metrics for each stage. The performance of the tool was then benchmarked using precision, recall, F1-score, and accuracy metrics, while the rewrite stage was evaluated using a classifier-based ambiguity-rate proxy and comparative ablation outputs. The findings indicate that Stage 1 (TF-IDF + LinearSVC) serves as an effective, lightweight gatekeeper, achieving a weighted F1-score of 82.10%. Stage 2 (DistilRoBERTa specialist) improved Non-Functional Requirement (NFR) subtype classification to 80.08%, compared to 74.71% for the linear baseline. Similarly, the Stage 3 ambiguity critic attained an F1-score of 83.91% using DistilRoBERTa, a significant increase over the 73.48% linear baseline. Finally, Stage 4 utilized a locally deployed Llama 3.1 model with the metadata-aware prompts captured from the previous stages, reducing the predicted ambiguity rate from 1.00 to 0.63, a -37 percentage-point reduction. These results demonstrate that a staged orchestration of models provides a high-performing and computationally efficient solution for automating the requirements analysis lifecycle.

Keywords: *Requirements Engineering, Requirements Analysis, Requirements Classification, Ambiguity Detection, Machine Learning, Large Language Models, Natural Language Processing, Software Development Life Cycle.*

TABLE OF CONTENTS

Authorship Declaration	i
Acknowledgments	ii
Abstract	iii
Table of Contents	iv
List of Abbreviations	vii
List of Figures	x
1 Introduction	1
1.1 Motivation	2
1.2 Aim and Objectives	2
1.3 Organization of Dissertation.	2
2 Background Research	3
2.1 Requirements Engineering: Systematic Literature Reviews.	3
2.2 ML-based Requirements Classification.	4
2.3 ML-based Ambiguity Detection	6
2.4 LLM-based Requirement Classification	7
2.5 Ambiguity Resolution techniques in Software Requirements	8
2.6 Critical Review of Related Work	9
2.7 Summary & Research Gap.	10
3 Methodology	11
3.1 Stage 0: Data Collection and Preparation	11
3.2 Stage 1 & 2: Machine Learning Classifier Development	12
3.3 Stage 3: Ambiguity Detection and Feature Extraction	12
3.4 Stage 4: Generative AI Enhancement	12
3.5 Evaluation 1: SVM Evaluation and Benchmarking	12
3.6 Evaluation 2: Full-Pipeline Performance	13
4 Feasibility Study	14
4.1 Technical and Data Feasibility	14
4.2 Experimental Setup and Implementation Details.	14
4.3 Pilot Study	15
5 Project Requirements	16
5.1 Functional Requirements	16
5.2 Non-Functional Requirements	17
5.3 Hardware and Software Requirements.	18
6 System Implementation & UI	19
6.1 Architecture and Service Composition.	19
6.2 PURR User Interface and Workflow	19
6.3 Runtime and Performance Controls	21

7 Data Profiling & Setup	22
7.1 Dataset Curation and Distribution	22
7.2 Preprocessing Pipeline and Split Protocol	23
7.3 Imbalance Handling with Class-Weighted and Oversampling Strategies . .	24
7.4 Prompt Design for the LLM Rewrite Stage	24
8 Experiment & Results	25
8.1 Stage 1: The Gatekeeper	25
8.2 Stage 2: The Specialist and Ablation Study	26
8.3 Stage 3: The Critic	27
8.4 Stage 4: The Rewriter and Prompt Ablation	28
9 Discussion	30
10 Conclusion	32
10.1 Research Contributions	32
10.2 Limitations	33
10.3 Scope for Future Work	33
References	34
A Project Management	36
A.1 Project Scope	36
A.2 Project Deliverables & Plan	36
A.3 Gantt Chart	38
A.4 Risk Management	40
A.5 Risk Prioritization Matrix	40
A.6 Risk Priority Number Threshold	41
A.7 Risk Management Table	41
A.8 PLES	43
B Generative AI	44
C Project Proposal and Research Changes	45
D Supplementary Material	46
D.1 Data Profiling	46
D.2 Stage 1 Supplementary Charts	47
D.3 Stage 2 Supplementary Charts	48
D.4 Stage 3 Supplementary Charts	50

List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
APA	Alignment with Perceived Ambiguity
API	Application Programming Interface
ARCP	Ambiguous Requirement Classification Pipeline
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
COM	Expert Comments
CSV	Comma-Separated Values
CTG	Controlled Text Generation
CPU	Central Processing Unit
DDR4	Double Data Rate 4
DT	Decision Tree
EHR	Electronic Health Record
FN	False Negative
FP	False Positive
FR	Functional Requirement
GenAI	Generative Artificial Intelligence
GPU	Graphics Processing Unit
GGUF	GPT-Generated Unified Format
GR	Golden Rewrite
GT	Ground Truth
GPT	Generative Pre-trained Transformer
GUI	Graphical User Interface
I	Impact
IBL	Inference-Based Learning
ICL	In-Context Learning
IEC	International Electrotechnical Commission

Table 1. List of Abbreviations (1/3)

Abbreviation	Definition
IEEE	Institute of Electrical and Electronics Engineers
ISO	International Organization for Standardization
KNN	K-Nearest Neighbors
L	Likelihood
LLM	Large Language Model
LLM's	Large Language Model (possessive form used in text)
LLMs	Large Language Models
ML	Machine Learning
NB	Naive Bayes
NFR	Non-Functional Requirement
NLG	Natural Language Generation
NLU	Natural Language Understanding
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
NR	Non-Requirement
NVME	Non-Volatile Memory Express
OSSD	Open Source Software Development
PDF	Portable Document Format
PII	Personally Identifiable Information
PLES	Professional, Legal, Ethical, and Social
PMI	Project Management Institute
POS	Part of Speech
PROMISE	Predictive Models in Software Engineering
PURE	Public Requirements Dataset
PURR	Pipeline-for-Understanding-Requirements-Reliability
QR	Query Rewrite
RA	Requirements Analysis
RAM	Random Access Memory

Table 2. List of Abbreviations (2/3)

Abbreviation	Definition
RE	Requirements Engineering
REQ	Requirements text
RF	Random Forest
RPN	Risk Priority Number
SBERT	Sentence-BERT
SDLC	Software Development Life Cycle
SSD	Solid-State Drive
SGD	Stochastic Gradient Descent
SLM	Small Language Model
SLR	Systematic Literature Review
SLRs	Systematic Literature Reviews
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
SMOTE	Synthetic Minority Over-sampling Technique
SRS	Software Requirements Specification
SVM	Support Vector Machine
SVC	Support Vector Classifier
TF-IDF	Term Frequency-Inverse Document Frequency
TN	True Negative
TP	True Positive
WSD	Word Sense Disambiguation

Table 3. List of Abbreviations (3/3)

LIST OF FIGURES

1	Distribution of RE phases studied in the SLR by (Cheng et al. 2024)	3
2	Distribution of RE phases studied in the SLR by (Zadenoori et al. 2025)	4
3	Proposed Experiment Workflow by Dev (2022)	5
4	Ambiguous Requirement Classification Pipeline (ARCP) by (Beqiri et al. 2024)	6
5	Prompt Design for classification tasks by (Zhang and Li 2024)	7
6	Our proposed Model	11
7	Initial PURR interface showing sidebar configuration, single-input panel, and batch CSV panel	20
8	Single requirement analysis appended to the persistent results table	20
9	Multi-row analysis table with rewrite content and CSV export capability	21
10	Dataset profile for the first 4,000 curated requirements used in the experiments	22
11	Precision-Recall curve for Stage 1 LinearSVC classifier	25
12	Ablation study on Stage 2: optimized LinearSVC vs fine-tuned DistilRoBERTa	26
13	Ablation study on Stage 3: LinearSVC vs DistilRoBERTa for ambiguity classification	27
14	Ablation study on Stage 4: Prompt variants for ambiguity reduction (lower is better)	28
15	Semester 1 Gantt Chart	38
16	Semester 2 Gantt Chart	39
17	Alternative dataset profiling view generated during pipeline data exploration	46
18	Stage 1 LinearSVC confusion matrix (FR vs NFR) from notebook evaluation outputs	47
19	Stage 2 optimized LinearSVC confusion matrix for 5-class NFR subtype classification	48
20	Stage 2 DistilRoBERTa confusion matrix for 5-class NFR subtype classification	49
21	Stage 2 ablation comparison chart (alternative notebook export)	49
22	Stage 3 LinearSVC confusion matrix for ambiguity classification	50
23	Stage 3 DistilRoBERTa confusion matrix for ambiguity classification	51

1 Introduction

In 2008, the opening of Heathrow Airport's Terminal 5 descended into chaos when its baggage handling system failed, misplacing a record 42,000 bags, and grounding countless flights. The root cause of the disaster was not a technical bug, but was a system designed by engineers who misunderstood operational complexities (Milmo 2008). This high-profile failure underscores a persistent crisis in software engineering; i.e. poorly defined requirements remain a leading cause of project failures (Larson 2014). A study by the Project Management Institute (PMI) supports this argument by indicating that 37% of project failures are a direct result of poor requirements' management, which leads to 47% of unmet project goals and the waste of 51% of project and program funding (Larson 2014; PMI).

In Requirements Engineering (RE), various stages take place to ensure that software systems meet user needs and business objectives. These include eliciting, analyzing, documenting, and validating requirements from stakeholders (Ahmad et al. 2021). The Software Requirements Specification (SRS) document is the outcome of these processes, establishing the contractual baseline between the stakeholders and developers (Rosado da Cruz and Cruz 2025). Yet despite this, the traditional approach to RE has struggled to keep pace with the increasing complexity and scale of modern software systems.

One of the stages in RE, Requirements Analysis (RA), is the critical phase where these requirements are refined, classified as Functional (FR) or Non-Functional (NFR), and validated for quality (Dev 2022; Rosado da Cruz and Ferreira Cruz 2025). This process is highly demanding and is often the bottleneck in the SDLC, but still critical in ensuring subsequent processes run smoothly (Budake et al. 2023). Furthermore, there is a high chance of undetected defects that can halt the entire SDLC process if left unaddressed (Beqiri et al. 2024). This often includes inherent ambiguity, meaning a sentence has more than one way of interpreting (e.g. "the system must be fast"). Ambiguity is what leads to misinterpretation by developers, which in some cases, can completely jeopardize the development of a system (like the Terminal 5 example) if left unaddressed. There are already existing approaches to resolve ambiguous requirements, often manual reviews and structured walkthroughs by domain experts, but are known to be resource intensive and not always error free (Izhar et al. 2025). Fortunately, there is an opportunity to enhance requirements analysis through automation (Budake et al. 2023; Hymel and Johnson 2025; Rosado da Cruz and Cruz 2025).

Machine Learning (ML) and recent advancements of Large Language Models (LLM) provide a complementary solution to key RA challenges (Cheng et al. 2024; Hymel and Johnson 2025). Classical ML approaches like Support Vector Machines (SVM) combined with Term Frequency-Inverse Document Frequency (TF-IDF) vectorization have been able to demonstrate high performance in ambiguity detection & requirements classification, one notable study achieving F1-scores of up to 0.826 (Dev 2022; Zhang and Li 2024). However, ML models often lack linguistic capabilities in handling language related tasks like ambiguity resolution or generating improved requirement rephrasings (Kim et al. 2024). Conversely, generative LLMs like GPT-4 and Llama 3 have shown significant results in

contextual reasoning and natural language understanding (Cheng et al. 2024; Kim et al. 2024; Tanjim et al. 2025), but lag behind and knowingly suffer from inconsistent inputs and lower precision in structured classification (Zhang and Li 2024). Therefore, there is an opportunity to systematically combine ML's classification reliability with LLMs language understanding in a unified hybrid framework.

1.1 Motivation

The motivation of this research is to address the gap between the use of ML and LLMs by proposing a hybrid ML-LLM pipeline which allows to leverage the performance of both domains. The motivation stems from 2 limitations observed from current research:

- The lack of Hybrid Architectures: Existing studies treat ML and GenAI as competing approaches rather than complementary tools (Dev 2022; Zhang and Li 2024).
- Limited exploration in Ambiguity Resolution: Studies have explored ambiguity detection using LLMs, but have often ignored more critical issues such as resolving the ambiguity (Bano 2015; Beqiri et al. 2024).

1.2 Aim and Objectives

The aim is to develop and evaluate a hybrid framework that combines ML classification with LLM-based ambiguity resolution to automate key Requirements Analysis tasks, benchmarking using manually annotated datasets. The research objectives are as follows:

- Identify existing research gaps for a hybrid classification and enhancement tool by reviewing existing studies in automating RA tasks using ML and LLMs.
- Design and implement a 2-stage framework using SVM for classification and term extraction, and a generative LLM for ambiguity resolution.
- Develop a software prototype of this framework, implementing the SVM classifier and integrating it with a prompt-engineered Llama 3.1 8B model.
- Evaluate the prototype's performance quantitatively against a manually annotated "ground truth" dataset, using Precision, Recall, F1-score, Accuracy, and rewrite-stage ablation indicators.

1.3 Organization of Dissertation

This dissertation is structured into 10 chapters. Following the Introduction and Background Research on RA automation (Chapters 1-2), Chapters 3-5 detail the Methodology, Technical Feasibility, and Project Requirements. Chapters 6-7 cover the System Implementation, architecture, and Data Profiling setup. The research concludes with Experimental Results (Chapters 8), followed by a critical Discussion and Conclusion (Chapters 9-10). Supporting appendices document project management and PLES.

2 Background Research

This chapter surveys existing approaches in automating RA tasks by focusing on three key activities: Requirements classification, ambiguity detection and ambiguity resolution. Section 2.1 will evaluate the research landscape by analyzing systematic literature reviews (SLR). Section 2.2 and Section 2.3 will examine the classical ML techniques, while Section 2.4 and Section 2.5 will discuss similar approaches with LLMs and resolving ambiguity. The chapter will conclude with a critical analysis in Section 2.7, allowing us to identify the gaps and motivation of the proposed hybrid framework.

2.1 Requirements Engineering: Systematic Literature Reviews

It is essential to understand the current research landscape in RE automation. Consequently, SLRs provide a way to summarize copious amounts of studies and identify trends, challenges, and gaps effectively. Zamani et al. (2021), for example, explored the trends and applications of ML in RE. By analyzing 65 studies, the author was able to identify ML research trends and application in automating RE analysis tasks. Furthermore, 12 out of the 65 studies applied ML techniques specifically in requirements classification, while 8 studies also explored ambiguity resolution using ML. The author also explored the different types of ML techniques, where SVM was the most used algorithm with 19 papers, followed by Naive Bayes (14 papers) and K-Nearest Neighbours (KNN) (11 papers). However, the author also mentioned the challenges researchers faced due to the lack of large, high-quality datasets, where some studies also suffered from over-fitting & under-fitting of their dataset. Other authors, Cheng et al. (2024) and Zadenoori et al. (2025), provided very recent SLRs identifying the current trends with GenAI for RE, indicating a shift in focus from classical ML to LLMs.

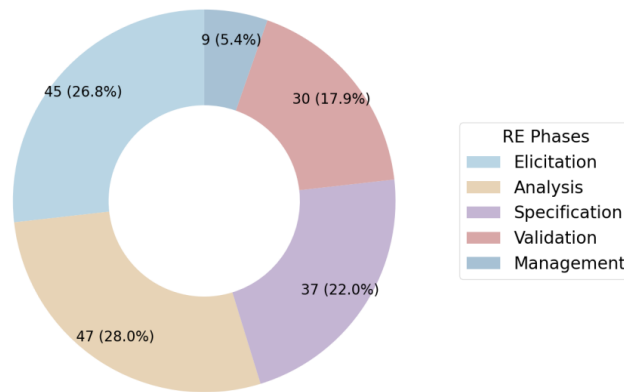


Fig. 1. Distribution of RE phases studied in the SLR by (Cheng et al. 2024)

Cheng et al. (2024) opened the field by analyzing 105 studies on GenAI in RE. The findings of their SLR revealed that OpenAI's GPT models dominated the field (67.3% of studies), where GPT-3 and GPT-4 were the most prevalent. The distribution graph in figure 1 shows RA as the most researched area (28%), followed by elicitation (26.8%) and specification

(22.0%). Cheng et al. (2024) also revealed that few-shot learning (40.8%) and zero-shot learning (36.8%) were the two most used prompt engineering strategies, as per the SLR. The author also identified persistent challenges common among the papers reviewed, where 61.9% of studies had issues with model interpretability, 52.4% with reproducibility, and 47.6% with controllability, indicating a potential gap in research.

Zadenoori et al. (2025) compiled a complementary SLR by analyzing 74 studies. The study shows while traditional natural language processing (NLP) research emphasized on defect detection and classification, LLM based research focused primarily on more complex tasks like elicitation and validation (22% of studies as shown in figure 2), consequentially contrasting the likes of Cheng et al. (2024)'s research where analysis dominates, suggesting a transition in creative tasks like elicitation matured to more object and measurable tasks like analysis. Zadenoori et al. (2025) also confirms GPT's dominance in RE (90% of the studies) and the prevalence of simple prompting techniques (zero-shot: 44%, few-shot: 29%), reinforcing that most research under-explores advanced fine-tuning methods.

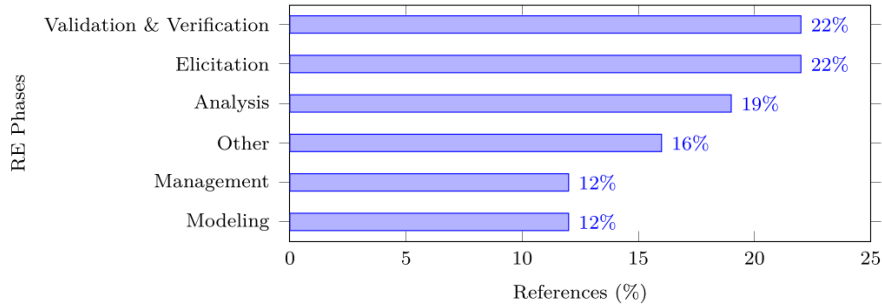


Fig. 2. Distribution of RE phases studied in the SLR by (Zadenoori et al. 2025)

Synthesizing both SLRs and (Zamani et al. 2021) confirms that (1) RA is a prime candidate for automation, (2) GPT models set the benchmarks for explainable LLMs, but alternatives remained unexplored, and (3) SVM remains the most reliable classical ML algorithm for structured classification tasks. Proving its reliability even with small to medium-sized datasets.

2.2 ML-based Requirements Classification

Classical ML approaches can provide a foundation in structuring requirements classification tasks (Zamani et al. 2021). An example of early work by Casamayor et al. (2009) was able to achieve this, by utilizing semi-supervised Naive Bayes classifiers in identifying non-functional requirements, while Wang et al. (2016) were able to demonstrate MapReduce-based Naive Bayes to classify customer requirements by processing levels. Other researchers however explored more diverse algorithms, such as Lu and Liang (2017), who compared Naive Bayes, J48, and Bagging, while Dev (2022) demonstrated high performance using SVM, SGD, and RF classifiers, achieving scores of up to 83.3% accuracy.

Both Lu and Liang (2017) and Dev (2022) addresses a shared problem with manual analysis of user app reviews to extract software requirements, which was said to be inefficient, and an infeasible task for developers. However, their approaches diverged significantly in their methodology, exploring different ML algorithms, and dataset compositions. Lu and Liang (2017) for example focused exclusively on user reviews, classifying them in 3 different categories FR, NFR with subtypes (reliability, usability, portability, performance) and "others", while Dev (2022) created their own novel hybrid dataset by combining formal requirements from SRS documents (PROMISE dataset) with user app reviews, extracted by (Hawari 2019), then classified them into three broader categories: FR, NFR, and Non-Requirements (NR).

Both authors explored different variations of ML classifiers, Dev (2022) compares SVM, SGD, and Random Forest combined with a standard TF-IDF vectorizer, as seen in an experimental pipeline in Fig. 3, while Lu and Liang (2017) compare Naive Bayes, J48, and Bagging, notably introducing a novel technique called AUR-BoW, which augments the short review texts using word2vec to improve feature representation. Conversely, both authors used 10-fold cross-validation to estimate the skill of the model on unseen data.

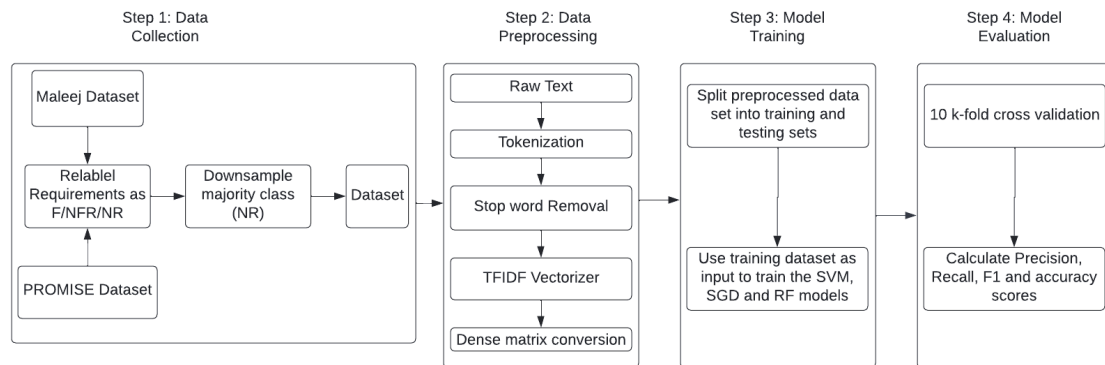


Fig. 3. Proposed Experiment Workflow by Dev (2022)

Therefore, both authors were successfully able to achieve high results in their model classifiers. Lu and Liang (2017) concluded that their novel AUR-BoW technique paired with the Bagging classifier yielded the best F1-Score of 71.8%. Dev (2022) found that the SGD classifier performed best on their hybrid dataset, achieving an 83% accuracy and a higher F1-score of 0.913 for NFR classification. Synthesizing both papers yield that ML is a valid approach to requirement classification, Lu and Liang (2017) demonstrates semantic augmentation is able to handle sparse, short texts, whereas Dev (2022) proves that models like SGD are robust enough to effectively classify requirements from a mixed-format hybrid dataset.

2.3 ML-based Ambiguity Detection

Challenges can still arise beyond classification, such as identifying linguistic ambiguity in requirements. A paper by Berry et al. (2003a) catalogued different types of ambiguity into types that are: 'lexical, syntactic, semantic, and pragmatic'. Building on this, Massey et al. (2014) developed a more revised taxonomy for regulatory and software requirements, providing explicit categories and subcategories to systematically identify ambiguous wording. This was then used to aid a case study on participants, who were able to detect 33.47 ambiguities in 104 lines of legal text requirements using the taxonomy developed. Shifting away from manual detection, some authors like Beqiri et al. (2024) and Izhar et al. (2025), focused specifically on automating ambiguity detection using ML techniques.

Beqiri et al. (2024) implemented a dual-input feature engineering approach, applying TF-IDF separately to the requirements and expert comments, sourced from industry experts, before aligning them. The author as such developed the "Ambiguous Requirement Classification Pipeline" (ARCP) in figure 4 combining both classification with AI explainability.

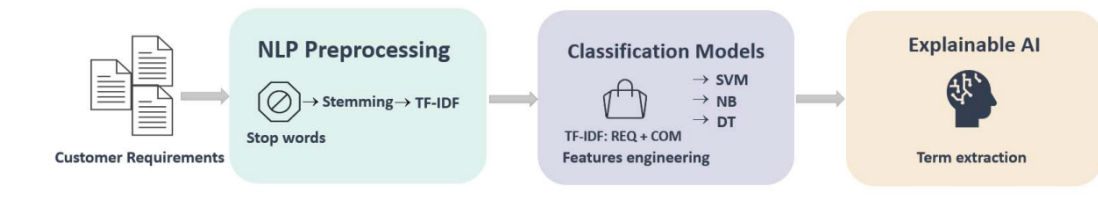


Fig. 4. Ambiguous Requirement Classification Pipeline (ARCP) by (Beqiri et al. 2024)

Conversely, Izhar et al. (2025) proposed a hybrid framework for general-domain requirements which first uses a rule-based model to catch simple, known ambiguities and then applies a machine learning model to identify more complex, contextual ones. Accordingly, both studies utilized classical ML models for the task, including SVM and Naive Bayes (NB), then employed TF-IDF for vectorization. Izhar et al. (2025) stepped further however by comparing wider baseline of models, including Logistic Regression and KNN and RF. Findings revealed that both authors were again able to achieve high performance in ambiguity detection. Izhar et al. (2025) concluded that their hybrid Random Forest classifier achieved the best results with an exceptional 97% accuracy and an F1-score of 0.92. While Beqiri et al. (2024) found that their SVM model performed best, but only when trained with the expert comments, achieving a high F1-score of 0.87 which a significant improvement over the 0.78 F1-score without comments. Therefore, both papers were able to demonstrate that ML is a valid approach to ambiguity detection in requirements. With Izhar et al. (2025) demonstrating a rule-based pre-filtering step provides highest performance in general-domain requirements, whereas Beqiri et al. (2024) proves that domain-specific expert comments can significantly enhance model performance in high-stakes railway requirements.

2.4 LLM-based Requirement Classification

Shifting away from classical ML models, the spark in GenAI, particularly LLMs complemented by their vast linguistic capabilities has provided new opportunities in requirements engineering (Cheng et al. 2024; Zadenoori et al. 2025). Early work by Tikayat Ray et al. (2023) proposed a novel tool they called *aeroBERT*, exploring the use of non-generative LLMs in requirement classification in the aerospace domain. Their results remained largely successful, achieving F1-scores of 0.83 in multiclass classification, largely due to their method of using a supervised fine-tuned BERT model. Other experimental studies, by Karlsson et al. (2025), challenged this by experimenting with two more models, GPT-4o and Llama 3.3-70B for a similar classification task. As a result, while having demonstrated consistent performance with F1 scores hovering around 0.80, Karlsson et al. (2025)'s models somewhat underperformed compared to the specialized fine-tuned non-generative models by Tikayat Ray et al. (2023). The authors Zhang and Li (2024) was able to one-up the comparisons by directly comparing the most popular generative LLMs in the context of binary classification of functional vs non-functional requirements.

Their experimental approach begins by evaluating six different LLMs, including GPT-3.5, GPT-4, GPT-4o, HuggingChat, Gemini, and DeepSeek, on both binary FR / NFR and multi-class NFR sub-category classification tasks. The authors then employed a technical method called in-context learning (ICL), which avoids parameter updates and instead relies on a specific prompt template (as shown in Fig. 5) and a verbalizer to map the model's single-token output (Z) to a target class.

Prompt Type	Prompt Template	Verbalizer
Binary Classification	"The requirement is [X]. Let's start: [Z]." Z= 1 (Functional), 0 (Non-Functional)	+1: Functional, 0: Non-Functional
Multi-Classification	"The requirement is [X]. Let's start: [Z]." Z=0-10 for subcategories	0-10: Maps to NFR subcategories(e.g., Performance, Usability)

Fig. 5. Prompt Design for classification tasks by (Zhang and Li 2024)

The output showed that different LLMs excelled in different tasks. Where HuggingChat excelled in the binary task, with an F1-score of 0.86 (rounded 2 decimal places), while GPT-4 led in the more complex multi-class task, achieving an F1-score of 0.57 (rounded 2 decimal places). Synthesizing the work along with Section 2.2, comparison of ML models vs non-generative LLMs, the results show that classical ML models still outperform generative LLMs especially in multiclass classification where Dev (2022) demonstrates high effectiveness of classical models like SGD on mixed-format data.

2.5 Ambiguity Resolution techniques in Software Requirements

While existing studies show that LLMs somewhat underperform (compared to their ML counterparts in classification tasks), new areas have remained unexplored, especially with LLMs linguistic capabilities in ambiguity resolution. A study by (Bano 2015) supports this statement by recognizing a significant gap in requirements engineering through the study of an SLR. They argued that vast majority of studies (81%) focused on detecting ambiguity rather than resolution. There has also been found a strong bias in research towards easier to solve syntactic ambiguity, where sentences have different meaning due to its grammatical structure, than solving more complex types like semantic and pragmatic ambiguity.

One related work however, addressed this gap by proposing an automated tool to specifically handle and resolve lexical and semantic ambiguity. The authors, Zait and Zarour (2018), argued that natural language in software requirements meet a critical problem with ambiguity which can lead to project failures, which must be caught on as early as possible, usually in the requirement elicitation phase. Accordingly, the author proposed a 3-step plan that combines NLP with semantic web techniques. They used NLP libraries such as Stanford Parser for Part-of-Speech tagging and sentence normalization. Second, it detects ambiguities by using BabelNet to find lexical ambiguities (aka words with multiple meanings) and a black-list of semantic terms (words like "all" or "fast") to find problematic sentences. Finally, it resolves these ambiguities using a graph-based method and Linked Data to determine the most correct meaning based on the word's context. The main limitation however was the lack of experiment results, leading to questions on how well their prototype actually performs in real world scenarios.

Other studies, however were able to demonstrate effective ambiguity resolution by involving a "human in the loop". One such author, Kim et al. (2024) focused on resolving conversational ambiguity by having the LLM generate clarifying questions to the user. Accordingly, they introduced a supervised Fine-Tuning pipeline called Alignment with Perceived Ambiguity (APA). The author was able to quantify the model's own uncertainty by measuring information gain (INFOGAIN) between the original query and its self-generated disambiguation, training the model to ask for clarification when ambiguity is high.

Other authors, Tanjim et al. (2025), proposed a novel Natural Language Understanding (NLU) + Natural Language Generation (NLG) framework called "Ambiguity-guided Query Rewrite". By detecting and classifying the ambiguity using a small language model, then using an LLM to rephrase the ambiguous query, the results were largely successful. They claimed that using the framework, the custom classifier achieving 92.16% accuracy, surpassing all baselines, including few-shot prompting of GPT-3.5-Turbo (72.68% accuracy) and Llama-3.1-70B (79.14% accuracy).

2.6 Critical Review of Related Work

This chapter compares the proposed hybrid pipeline against representative studies analyzed in the background research. To keep the comparison consistent, each study is evaluated by the same dimensions, that is task scope, modelling strategy, evidence strength, and practical constraints for deployment. The selected works span classical ML baselines (Dev 2022; Lu and Liang 2017), explainable ambiguity detection (Beqiri et al. 2024; Izhar et al. 2025), LLM-based classification (Zhang and Li 2024), and ambiguity-aware generative methods (Kim et al. 2024; Tanjim et al. 2025), with broader trend context from a recent SLR (Cheng et al. 2024).

Table 4. Comparison with state-of-the-art studies

Study	Primary focus and method	Reported evidence
Dev (2022)	Requirement classification from mixed app reviews + requirements artifacts using TF-IDF with SVM/SGD/RF.	Best model (SGD) reported 83% accuracy with 10-fold cross-validation.
Lu and Liang (2017)	Automatic NFR classification from augmented app reviews using AUR-BoW with Naive Bayes/J48/Bagging.	Best combination (AUR-BoW + Bagging): precision 71.4%, recall 72.3%, F-measure 71.8%.
Beqiri et al. (2024)	Explainable ambiguity classification in rail requirements using SVM/DT/NB with expert comments as auxiliary features.	Up to F-score 0.87 and recall 0.88 (with expert comments); lower without comments.
Izhar et al. (2025)	Hybrid ambiguity detector combining rule-based lexical filtering with Random Forest classification.	Reported 97% accuracy, 97% precision, 89% recall, and 92% F1 on a 1,553-requirement dataset.
Zhang and Li (2024)	LLM-only requirement classification using in-context prompts across GPT/HuggingChat/Gemini/DeepSeek.	Binary best: HuggingChat (85.64% accuracy, 86.17% F1); multi-class best: GPT-4 (65.97% accuracy, 57.14% F1).
Tanjim et al. (2025)	Ambiguity-guided query rewrite framework (NLU + NLG), rewriting only predicted ambiguous inputs.	Detection model outperformed few-shot LLM baselines; framework improved rewrite robustness using ambiguity-guided routing.
Kim et al. (2024)	Alignment with Perceived Ambiguity (APA), training LLMs to request clarification under uncertain semantics.	Across AmbigQA-style benchmarks, APA variants improved ambiguity-focused scores and reduced misaligned clarifications.
Cheng et al. (2024)	Systematic literature review of GenAI in RE (105 studies, 2019–2024).	GPT models dominated (67.3%); recurring challenges include interpretability (61.9%), reproducibility (52.4%), controllability (47.6%).

The comparison reveals several consistent patterns. Classical ML-centric studies remain strong for structured classification and ambiguity detection, particularly when the task is narrow and labels are well defined (Beqiri et al. 2024; Dev 2022; Izhar et al. 2025). Similarly, LLM-centric studies offer stronger linguistic flexibility and ambiguity handling, but can often trade off controllability, reproducibility, and evaluation consistency (Cheng et al. 2024; Kim et al. 2024; Tanjim et al. 2025; Zhang and Li 2024). In practical terms, the literature supports complementarity rather than replacement.

2.7 Summary & Research Gap

Summarizing the literature, it is clear that there has been a divergence in approaching requirement analysis automation. On one hand, classical ML models demonstrated high reliability in structured classification. Section 2.2 covered ML based approaches in classifying functional and non-functional requirements, where Lu and Liang (2017) and Dev (2022) were able to achieve F1-scores well above 0.80 using RF and SGD classifiers respectively, proving their robustness even with mixed-format datasets. Section 2.3 showcased similar success with SVM classifiers in ambiguity detection, such as the work by Beqiri et al. (2024) who were able to classify ambiguity with the help of domain specific expert comments, achieving results of up to 0.87. Izhar et al. (2025) also showed success in ambiguity detection but with the help of a rule based system to catch simple ambiguous words.

On the other hand, recent studies on generative LLMs often underperform to classical ML in these same discrete classification tasks, exhibiting lower precision and consistency. Zhang and Li (2024) covered in section 2.4, demonstrated low multiclass classification performance using LLMs, all receiving F1-ranges around 0.5 to 0.6. This divergence presents a significant research gap, which this dissertation aims to address, where existing studies treat ML and LLM approaches as competing rather than complementary tools (Zhang and Li 2024). There is also a distinct lack of research exploring hybrid architectures that leverage these technologies as complementary components.

In the next section we will cover our proposed methodology to address this gap by developing a hybrid pipeline that combines the structured reliability of ML classifiers with the linguistic prowess of LLMs for ambiguity resolution.

3 Methodology

Fig 7 shows a prototype pipeline, divided into 4 distinct phases that aligns with the project's core objectives:

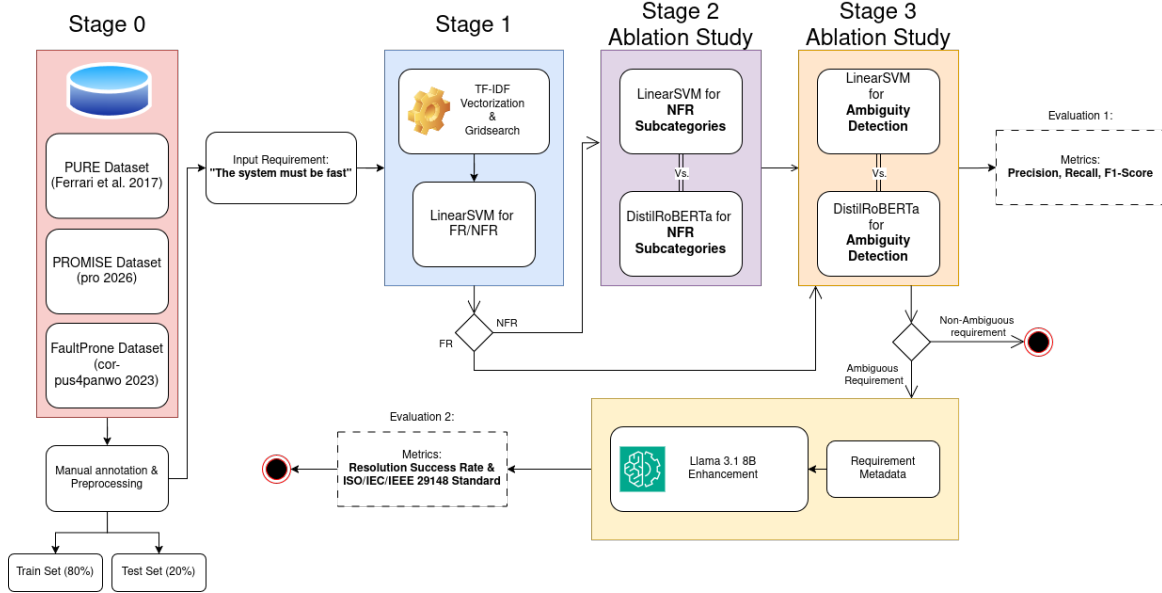


Fig. 6. Our proposed Model

This research will adapt a controlled experiment to evaluate the hybrid ML-LLM pipeline for requirements analysis. Therefore, 4 stages have been structured as followed:

3.1 Stage 0: Data Collection and Preparation

The requirement dataset will aggregate from the PUBLIC REquirements dataset (PURE) (Ferrari et al. 2017), PROMISE dataset (pro 2026) and the Fault Prone SRS Dataset (corpus4panwo 2023) into one combined corpus. These datasets were mainly chosen due to its ethical compliance, as well as its diversity in different domains of software engineering.

3.1.1 Stage 0.1: Preprocessing. For this experiment, the pipeline will use the first 4,000 entries from the verified integrated corpus (final_corpus_verified - Sheet1.csv, n rows=4000). Requirements were normalized via lowercasing, removal of bullet-style numbering, numeric token masking (numtoken), punctuation stripping, and strict de-duplication on cleaned text. For Stage 1 and Stage 2, lemmatization and domain-specific stopword filtering were applied; rows with fewer than 3 cleaned tokens were removed.

3.1.2 Stage 0.2: Train-Test Split Strategy (Split-First Protocol). To prevent data leakage, an 80%/20% stratified split protocol was used with stage-specific balancing. In Stage 2, the main LinearSVC setup used class-weighted optimization on the training split (no synthetic data in test), with additional quick-fix ablations comparing SMOTE and RandomOverSampler

on training only. In Stage 3, class balancing used downsampling before split to produce balanced ambiguous/non-ambiguous classes.

3.2 Stage 1 & 2: Machine Learning Classifier Development

This stage focuses on training the Stage 1 gatekeeper and the Stage 2 specialist model family. Stage 1 uses a LinearSVC gatekeeper over TF-IDF features (n-grams up to bigrams, `max_df=0.85`, `min_df=3`) for FR/NFR routing. Stage 2 uses two competing pipelines for ablation: (i) a class-weighted LinearSVC baseline over TF-IDF (1-3 grams, `min_df=1`) with 5-fold GridSearchCV over $C \in \{0.1, 0.5, 1.0, 5.0, 10.0\}$, plus quick-fix variants with SMOTE and RandomOverSampler; and (ii) a fine-tuned DistilRoBERTa specialist (`distilroberta-base`, up to 6 epochs with early stopping, learning rate $2e-5$, train batch size 8, eval batch size 16, `max_length=192`, weighted loss).

3.3 Stage 3: Ambiguity Detection and Feature Extraction

For Stage 3, ambiguity detection uses a task-specific preprocessing path that intentionally retains stopwords to preserve cues such as modal uncertainty. Two classifiers are again compared through ablation: a TF-IDF + LinearSVC baseline (1-3 grams, GridSearchCV over $C \in \{0.1, 0.5, 1.0, 5.0, 10.0\}$) and a DistilRoBERTa binary classifier (5 epochs, learning rate $2e-5$, batch size 16, `max_length=128`). This stage validates that contextual deep models better capture ambiguity semantics than linear boundaries alone.

3.4 Stage 4: Generative AI Enhancement

We employ the Llama 3.1 8B model using metadata-aware prompting to ensure deterministic and high-quality outputs. The prompt is dynamically constructed using the original requirement plus upstream metadata (predicted class, predicted subtype, and predicted ambiguity label). Four prompt variants were evaluated (`plain`, `ambiguity_only`, `metadata`, `metadata_rules`) to compare context depth while preserving original intent.

3.5 Evaluation 1: SVM Evaluation and Benchmarking

The final phase evaluates the classifiers against the held-out ground-truth test set. The SVM is measured using precision, recall, and F1-score, which together show whether the model is useful as a requirements triage tool rather than just a technical classifier. In practical terms, these metrics indicate whether the pipeline catches ambiguous requirements reliably, avoids unnecessary false alarms, and keeps the review workload manageable.

- True Positive (TP): The model correctly predicts a requirement is Ambiguous.
- False Positive (FP): The model incorrectly predicts a requirement is Ambiguous.
- False Negative (FN): The model fails to predict a requirement is Ambiguous.

3.5.1 Precision. As identified in EQ-1, precision measures how often a requirement predicted as ambiguous is actually ambiguous. High precision matters because it limits false alarms, which means the LLM is not asked to rewrite requirements that are already clear.

For a software engineering team, this keeps the automation focused and prevents time being spent on unnecessary rework.

$$\text{Precision} = \frac{\text{True Positive (TP)}}{\text{True Positive (TP)} + \text{False Positive (FP)}} \quad (\text{EQ-1})$$

3.5.2 Recall. As identified in EQ-2, recall measures how many of the truly ambiguous requirements are caught by the classifier. High recall is important because missed ambiguous statements stay hidden in the specification and can later become defects, test gaps, or development misunderstandings. In practical terms, recall shows whether the tool is good at finding the requirements that really need human attention.

$$\text{Recall} = \frac{\text{True Positive (TP)}}{\text{True Positive (TP)} + \text{False Negative (FN)}} \quad (\text{EQ-2})$$

3.5.3 F1-Score. As identified in EQ-3, F1-score summarizes the balance between precision and recall in a single value. A high F1-score means the classifier is not only catching the right requirements, but doing so without overwhelming the team with false positives. This makes it a useful measure for comparing the pipeline against related studies such as Dev (2022) and Beqiri et al. (2024).

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{EQ-3})$$

3.6 Evaluation 2: Full-Pipeline Performance

The second part of the evaluation measures the quality of rewritten requirements produced by the Phase 4 LLM using an automatic ambiguity-rate proxy. Rewrites from each prompt variant are re-classified by the Stage 3 ambiguity model, and the fraction still predicted as ambiguous is compared across variants. To make the ablation interpretable, we also compute a baseline ambiguity rate on the same candidate set before rewriting and report ambiguity reduction as baseline minus post-rewrite rate. This is paired with average rewrite length to monitor over-expansion. For a software engineering team, this gives a pragmatic first-pass measure of whether rewrites are likely clearer, while preserving a future extension point for human judging and semantic metrics.

4 Feasibility Study

This chapter proposes the feasibility of the project by outlining the technical requirements, tools needed and preprocessing steps to be carried out before the implementation of the prototype tool. This evidence is to show the practicality of the proposed methodology in Section 3, and the feasibility will be tested in a pilot study outlined in 4.2.

4.1 Technical and Data Feasibility

The components required for this dissertation are all publicly available and open-source, ensuring that experiment reproducibility is achievable. Furthermore, the key technologies listed below have been tested and validated:

4.1.1 Data Source. The experiment uses three public requirements datasets: PURE Dataset (Ferrari et al. 2017), PROMISE (pro 2026), and Fault Prone SRS Dataset (corpus4panwo 2023), which will then be adapted into combined corpus using python scripting. Each dataset has been sourced from Kaggle repositories along with appropriate citations ensuring data integrity and ethical use.

4.1.2 ML Classifier. The classification stages are built in Python 3 (PYTHON 2025) using scikit-learn (Scikit-learn 2024), a standard library for text classification.

4.1.3 Generative Model. The Llama 3.1 8B model, accessed via hugging face transformers library, will be used as the ambiguity resolution component. This model was chosen by its size-to-performance ratio and its open-source nature.

4.1.4 Development and Computational Resources. The prototype is developed in Jupyter Notebook. For the heavier transformer and generation stages, Google Colab with GPU will be used to complete the experiments within a reasonable timeframe.

4.2 Experimental Setup and Implementation Details

The updated experiments ran from `PURR/pipeline.ipynb` on a Lenovo ThinkPad T14 AMD Gen 1 and a Google Colab T4 GPU. A Python notebook environment with scikit-learn and NLTK supports the TF-IDF and LinearSVC stages; DistilRoBERTa acts as the compact transformer specialist for the classification stages; and local Llama 3.1 8B in GGUF format performs the rewrites. The dataset protocol uses an 80/20 validation scheme with stage-specific balancing as implemented in the notebook.

4.3 Pilot Study

A small pilot study confirms that the workflow can be reproduced end to end. The example below shows how a vague requirement can be classified, how the ambiguous term is isolated, and how the rewrite stage turns that input into a more measurable statement.

```
1 domain = "Healthcare"
2 requirement_type = "Non-functional"
3 ambiguous_req = "The EHR must load patient records quickly."
4 extracted_term = "quickly"
5
6 # Prompt uses requirement_type, domain, and extracted_term
7 prompt_text = (
8     f"You are refining a {requirement_type} requirement in the {domain} domain. The original requirement contains the ambiguous term '{extracted_term}'. Rewrite it so it is specific, measurable, and verifiable. Output only the final rewritten requirement with no labels, quotes, or explanations. Original requirement: [{ambiguous_req}]"
9 )
```

Code Listing 1. Simulated SVM output and LLM prompt

Output: "The EHR system shall load patient records within 5 seconds or less from the time the user initiates the load process, 95% of the time, during peak usage hours (7am-5pm, Monday-Friday)"

The prompt is built from the classifier output and the original requirement text. In this example, the SVM identifies the word "quickly" as the source of ambiguity, and the LLM converts it into a statement that is specific enough to test. The generation settings are kept low and stable so that the output stays coherent across runs. The practical takeaway is that the pipeline can turn a vague specification into a requirement that a software team can estimate, implement, and verify with less manual rewriting.

5 Project Requirements

This chapter outlines the system requirements needed for the development of this prototype, as we design full fledged web application named P.U.R.R (Pipeline-for-Understanding-Requirements-Reliability). This is to ensure that the proposed hybrid machine learning and LLM pipeline can be demonstrated as a practical Software Engineering tool. The prototype requirements will be categorized as followed: Functional, Non-Functional, and Hardware/Software requirements.

5.1 Functional Requirements

In order to satisfy the core objectives of this project, we define a set of functional requirements that outline the specific user objectives that would be expected as if the system were to be used in a real-world software engineering context. Table 5 lists these requirements as followed:

Table 5. System Functional Requirements (FR)

ID	Description
FR1	The system shall allow users to input raw software requirement text via a user interface.
FR2	The system shall pass the input through a Stage 1 Gatekeeper (LinearSVC) to classify the text as either a Functional or Non-Functional Requirement.
FR3	If classified as Non-Functional, the system shall route the text to a Stage 2 Specialist (DistilRoBERTa) to predict its specific subtype (Security, Usability, Performance, Maintainability, Availability).
FR4	The system shall utilize a Stage 3 Critic model to flag if the requirement contains linguistic ambiguity.
FR5	If ambiguity is detected, the system shall automatically prompt a Stage 4 Generative LLM to rewrite the text to comply with ISO/IEC/IEEE 29148 measurability standards.
FR6	The system shall display the final classifications, ambiguity labels, and the generated rewrite clearly to the user in the form of a table.

5.2 Non-Functional Requirements

Non-functional requirements are not necessary to the systems core functionality, but defining them is important to ensure the system is practical and usable in a real-world software engineering context. Table 6 lists these requirements as followed:

Table 6. System Non-Functional Requirements (NFR)

ID	Description
NFR1 (Privacy)	The system must execute all Machine Learning and Large Language Model inferences entirely locally on the host machine to prevent the leakage of sensitive corporate specifications to external APIs.
NFR2 (Performance)	The Stage 1 to Stage 3 classification pipeline must return inferences to the UI in under 2 seconds to ensure a seamless user experience.
NFR3 (Efficiency)	The 8-Billion parameter generative LLM must be quantized (GGUF format) and cached in RAM to allow execution on a standard CPU without system failure.
NFR4 (Reliability)	The generative LLM must be constrained by strict prompt templates and a low temperature parameter (e.g., $T = 0.0$) to ensure deterministic, repeatable text generation and prevent AI hallucinations.
NFR5 (Usability)	The GUI must be intuitive, requiring no prior machine learning expertise from the software engineer utilizing the tool.

5.3 Hardware and Software Requirements

Google Colab and local CPU execution were used depending on the constraint of each stage. For instance, the classic ML training and evaluation stages were executed locally on the CPU, while the GGUF Llama inference was executed using google colab's proprietary T4 GPU. The specific hardware and software environment are detailed in Table 7 as followed:

Table 7. Hardware and Software Environment

Category	Specification
Processor (CPU)	AMD Ryzen 5 PRO 4650U (6 Cores / 12 Threads)
Memory (RAM)	32 GB DDR4 (Required for caching the 8B LLM model weights)
Storage	NVMe SSD (For rapid model loading and data retrieval)
Graphics (GPU)	Google Proprietary T4 GPU
Operating System	Windows 10/11 or Linux (Ubuntu 22.04 LTS)
Programming Lang.	Python 3.x
Core ML Libraries	scikit-learn, imbalanced-learn, nltk
Deep Learning Libs	Hugging Face transformers, torch
LLM Engine	llama-cpp-python (For GGUF 8-bit quantization support)
Frontend Framework	Streamlit (With @st.cache_resource implementation)

6 System Implementation & UI

This chapter outlines the implementation of the pipeline experiment as a real world, modular service-based system. We will also look into how the user interface supports the practical requirement triage. Finally, the implementation was intentionally separated into a light weight frontend client and a backend orchestrational layer so that the model component can be updated independently without affecting the interface.

6.1 Architecture and Service Composition

The PURR prototype interface was implemented with Streamlit as the user presentation layer and FastAPI as the backend application layer. During application startup, the backend loads and initializes the model pipeline, then exposes the endpoints into 3 distinct analysis tools: single analysis, batch analysis, and PDF knowledge-base ingestion.

Table 8. Core backend modules and implementation roles

Module	Role in pipeline execution
ModelService	Loads dataset artefacts and executes Stage 1 classical predictions, including confidence and explainable terms.
TransformerService	Executes transformer-based subtype and ambiguity predictions when checkpoints are available.
RagService	Ingests PDF content and retrieves top- k context snippets for analysis-time grounding.
RewriteService	Runs local Llama GGUF inference for controlled requirement rewriting.
AnalysisService	Orchestrates end-to-end request flow and returns unified results to API endpoints.

Table 8 shows the core backend modules and their roles in the pipeline execution. By using a modular approach we can allow for independent updates to each component, such as swapping out the transformer model or adjusting the rewrite logic without requiring changes to the frontend service.

6.2 PURR User Interface and Workflow

The interface follows a table workflow structure. The user written requirement is appended as a row into the table, this supports continuous review sessions where software analysts can compare multiple requirements, confidence scores, ambiguity flags, and rewrites in one view.

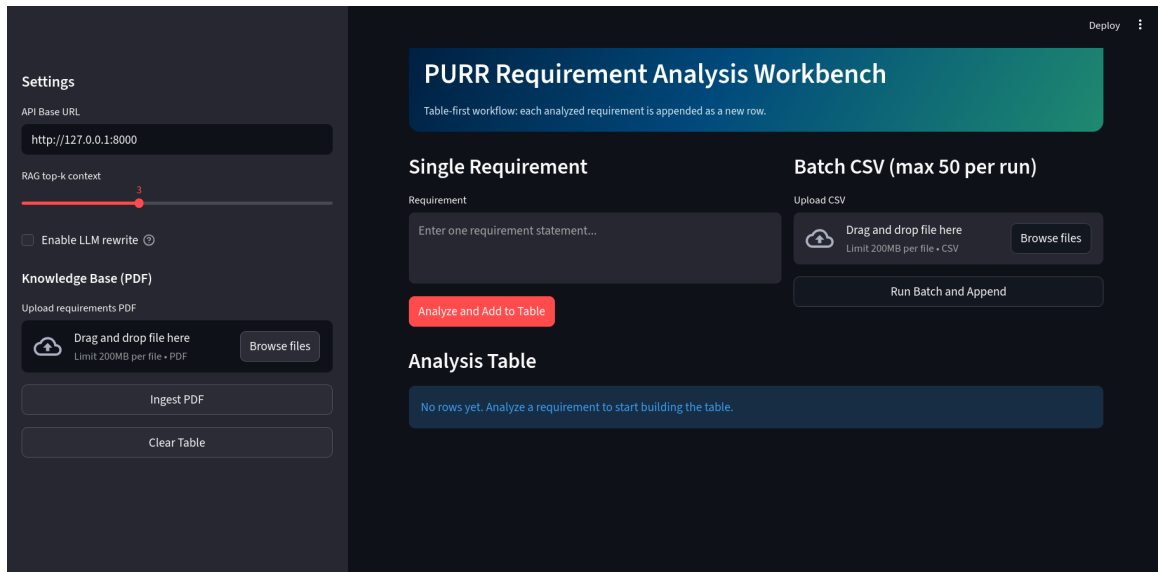


Fig. 7. Initial PURR interface showing sidebar configuration, single-input panel, and batch CSV panel

As shown in Figure 7, the side panel allows for the user to configure the API endpoint and retrieval depth, and optionally able to input a PDF knowledge source, which will further improve LLM rewrite quality, and submit either one requirement or a CSV batch (up to the configured batch limit).

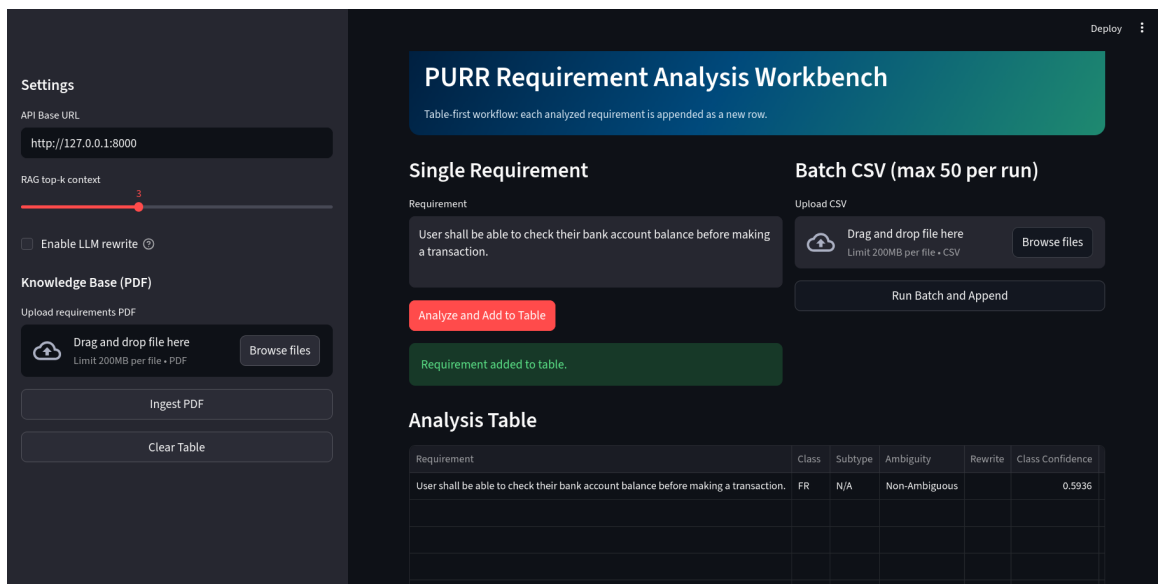
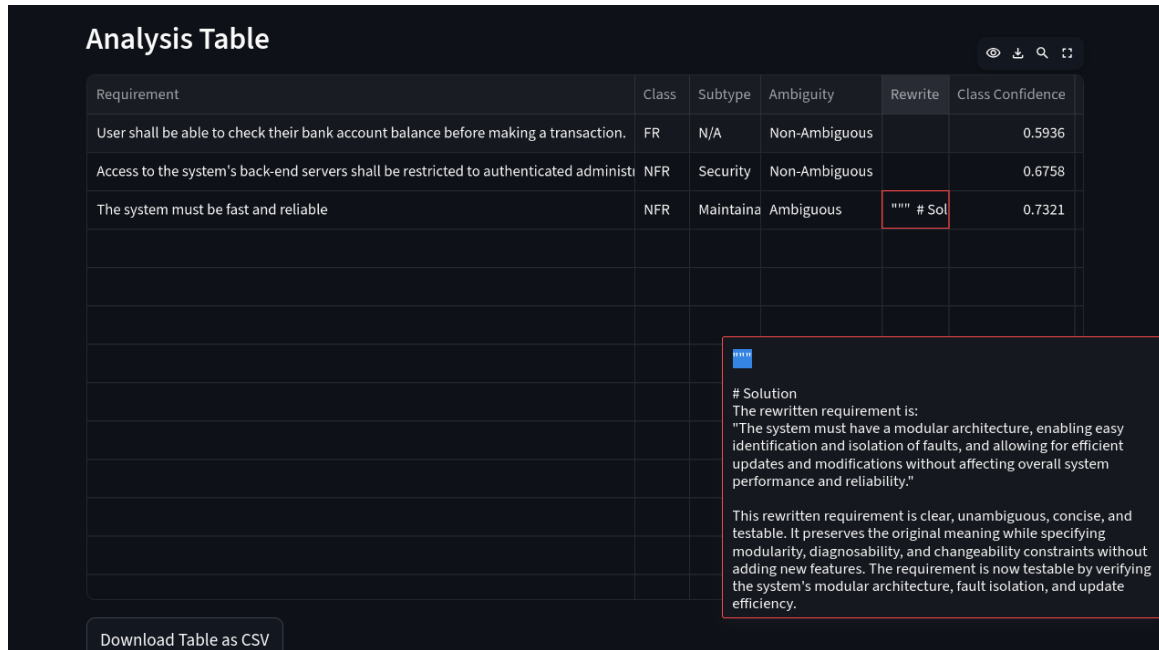


Fig. 8. Single requirement analysis appended to the persistent results table

Figure 8 shows the output in the analysis Table after a single requirement is analysed. The table records predicted class, subtype, ambiguity, and confidence values in a format suitable for iterative triage.



Requirement	Class	Subtype	Ambiguity	Rewrite	Class Confidence
User shall be able to check their bank account balance before making a transaction.	FR	N/A	Non-Ambiguous		0.5936
Access to the system's back-end servers shall be restricted to authenticated administ	NFR	Security	Non-Ambiguous		0.6758
The system must be fast and reliable	NFR	Maintaina	Ambiguous	"#" # Sol	0.7321

Solution

The rewritten requirement is:
 "The system must have a modular architecture, enabling easy identification and isolation of faults, and allowing for efficient updates and modifications without affecting overall system performance and reliability."

This rewritten requirement is clear, unambiguous, concise, and testable. It preserves the original meaning while specifying modularity, diagnosability, and changeability constraints without adding new features. The requirement is now testable by verifying the system's modular architecture, fault isolation, and update efficiency.

Download Table as CSV

Fig. 9. Multi-row analysis table with rewrite content and CSV export capability

Figure 9 shows the cumulative review state after multiple entries, including rewritten output for ambiguous requirements and final export of the table as CSV.

6.3 Runtime and Performance Controls

As mentioned in section 5.3, limitations in hardware had to be taken into account, therefore runtime controls were introduced to keep local execution in a CPU only environment. The heavy llama model assets were also loaded once during backend startup, avoiding the repeated initialization overhead. On the frontend, analysis rows are preserved in session state, which minimizes redundant recomputation during user interaction.

Furthermore, computationally intensive generation is gated by a toggle button in the user interface, which allows the user to control when the rewrite generation is invoked. This is typically only needed when ambiguity has been detected, which allows to keep classical ML classification fast while reserving the higher-cost LLM rewrites for cases where quality improvement is needed.

7 Data Profiling & Setup

In this chapter, we profile the combined corpus used by the hybrid pipeline and document the exact decisions and setup used to provide reproducibility. The first 4000 rows of the combined corpus were curated and verified, which is then used consistently as a new dataset across all stages, matching the implementation in `pipeline.ipynb`. The labels were retained for requirement class (FR/NFR), subtype, and ambiguity, enabling a shared source for both the classification and LLM rewrite experiments.

7.1 Dataset Curation and Distribution

The working corpus was curated by 3 public datasets as mentioned in Section 3.1 ((Ferrari et al. 2017), (pro 2026), and (corpus4panwo 2023)), which were then consolidated into a single verified sheet. Using python scripting, we then standardized column structure and label naming. As a result, the first 4,000 entries included 2,597 FR labels and 1,403 NFR labels. For ambiguity, 3,415 entries were labeled Non-Ambiguous and 585 were labeled Ambiguous.

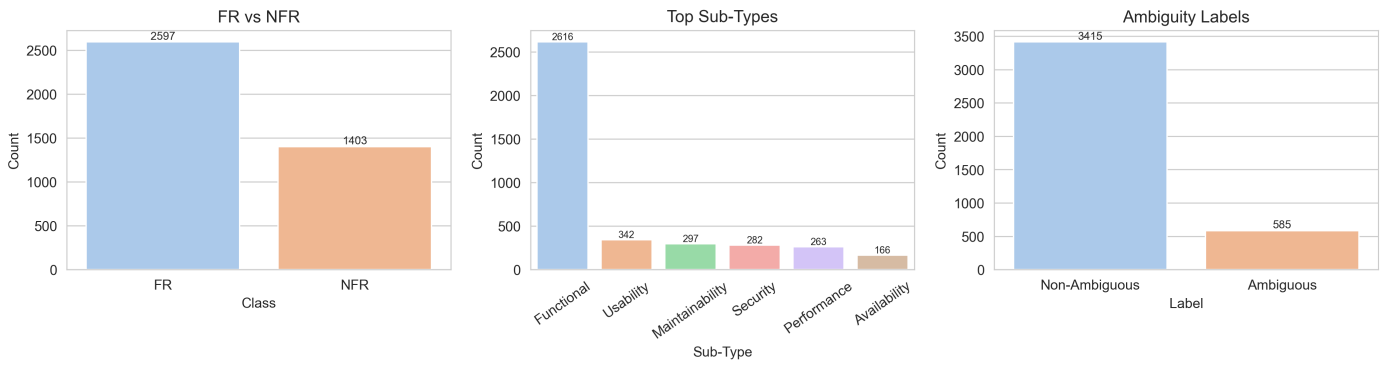


Fig. 10. Dataset profile for the first 4,000 curated requirements used in the experiments

Figure 10 shows that the corpus is naturally imbalanced across both class and ambiguity labels, motivating separate imbalance handling strategies at different stages of the pipeline.

Table 9. Core distribution statistics for the 4,000-row working corpus

Label Group	Category	Count
Class	FR	2,597
Class	NFR	1,403
Ambiguity	Non-Ambiguous	3,415
Ambiguity	Ambiguous	585

For Stage 2 subtype modelling, the five target NFR categories contained 1,350 rows before text cleaning: Usability (342), Maintainability (297), Security (282), Performance (263), and Availability (166).

Table 10. Example entries from the curated dataset

ID	Requirement Text (excerpt)	Class	Sub-Type	Ambiguity
1	The app shall run on a smart phone with Android OS least version 2.3.	NFR	Maintainability	Non-Ambiguous
2	All layout shall be according to the TU/e corporate identity, unless in conflict with requirements UCOR003-UCOR006.	NFR	Usability	Non-Ambiguous
3	The campus map shall cover the campus of the TU/e.	FR	Functional	Non-Ambiguous
...	...	...	...	...

7.2 Preprocessing Pipeline and Split Protocol

The preprocessing steps followed best practices for classical text classification models while also considering the needs of later stages. For stage 1 and 2, this involved lowercasing of text, removal of leading bullet numbering, stripping punctuation and lemmatization + domain stopword filtering were applied. Ajeigbe and Maddog (2021) states that lemmatization is preferred over stemming, as it preserves more of the original word form, which in turn facilitates better explainability and rewrite capabilities, but also mentioned the increased cost of computational resources. Finally records with fewer than three cleaned tokens were removed to filter out noise, and exact duplicates were dropped based on the cleaned text to ensure a more robust training set.

This produced the following counts for Stage 1: 3,956 rows after minimum-length filtering, 3,872 after deduplication, and class counts of FR=2,518 and NFR=1,354 before balancing. Majority downsampling was then applied to form a balanced set of 2,708 rows (1,354 per class), followed by an 80/20 split (2,166 train, 542 test).

For Stage 2, the five-class NFR subset reduced from 1,350 to 1,336 after length filtering and to 1,304 after deduplication, then split 80/20 into 1,043 training and 261 test rows. A split-first protocol was used to avoid leakage; therefore, no synthetic examples were introduced into the held-out test partition.

Unlike stage 1 and 2, stopwords were intentionally retained for Stage 3 ambiguity detection to preserve key signals such as modal uncertainty. Recent studies by (Kaur and Bhatia 2025) demonstrate that detecting ambiguity relies heavily on specific lexical and syntactic features. Preserving these structural cues, such as modal uncertainty (e.g., should, may, could), was therefore needed for this stage. This decision yielded 3,999 rows after length filtering and 3,944 after deduplication, with class counts Non-Ambiguous=3,370 and Ambiguous=574 before balancing. Downsampling produced a balanced set of 1,148 rows (574 per class), then an 80/20 split (918 train, 230 test).

7.3 Imbalance Handling with Class-Weighted and Oversampling Strategies

Imbalance was handled differently per stage based on task characteristics. For the Stage 2 TF-IDF + LinearSVC main setup, class-weighted optimization (`class_weight='balanced'`) was used on the training fold without synthetic interpolation.

To verify robustness, additional quick-fix ablations were implemented with SMOTE and RandomOverSampler alongside class-weighted LinearSVC. This allowed direct comparison between synthetic interpolation, random duplication, and cost-sensitive learning. The same split was retained across variants to ensure fair comparison.

7.4 Prompt Design for the LLM Rewrite Stage

In the rewrite stage 4, we used a local GGUF (GPT-Generated Unified Format) version of Llama 3.1 8B. Its design to be a metadata-aware generation task rather than free-form rewriting allows the model to leverage the predicted class (FR/NFR), predicted subtype (for NFR), and predicted ambiguity from earlier stages to produce more targeted rewrites. This is also known as CTG (Controlled Text Generation), where conditioning the prompt on specific metadata variables such as our predicted class and subtype allows the model to constrain its output to desired attributes, which in turn reduces hallucination (Keskar et al. 2019; Liu et al. 2021). Rewriting was restricted by default to requirements predicted as ambiguous (`rewrite_only_ambiguous=True`) to reduce unnecessary modification of already clear statements.

An ablation over four prompt variants was implemented: `plain`, `ambiguity_only`, `metadata`, and `metadata_rules`. The final variant injected subtype-specific guidance (e.g., measurable constraints for Performance or explicit controls for Security) while preserving the original intent, successfully adapting ambiguity-guided rewriting frameworks (Tanjim et al. 2025) to software specification standards. Outputs were saved side-by-side in `llm_rewrites_ablation.csv` for comparative analysis.

8 Experiment & Results

In this chapter, we will discuss the experimental design and outcomes of the proposed 4-stage hybrid pipeline. We will then compare the results with related work to contextualize the performance of our models. Finally, the results will be synthesized across the stages to show how the pipeline components work together to improve requirement analysis.

8.1 Stage 1: The Gatekeeper

Stage 1 ran a grid search over multiple unique iterations in order to find the best settings for the TF-IDF vectorizer and the LinearSVC classifier. Using this method, the final configuration was found where TF-IDF with ngram range up to bigrams, a maximum document frequency of 0.85, and finally a minimum document frequency of 3, provided the best performance for the classifier. Standard preprocessing discussed in the previous section were applied beforehand, and majority downsampling was applied to form a balanced FR/NFR set. The purpose of this stage was to check whether a lightweight first-pass filter can route most requirements correctly before a human engineer has to inspect them.

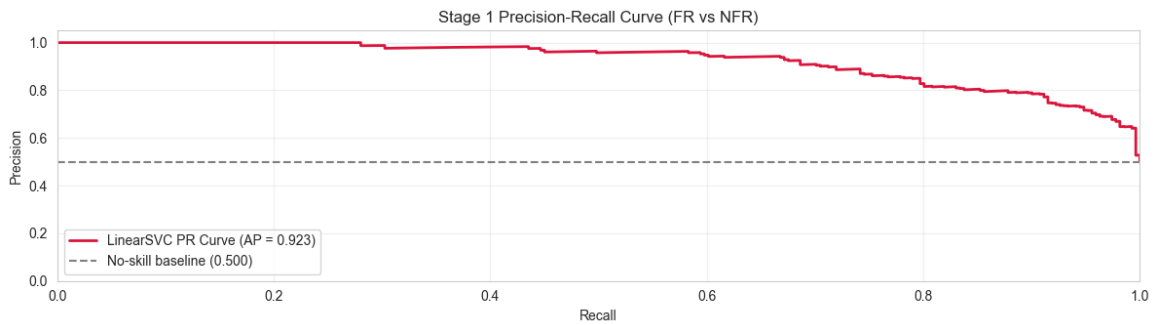


Fig. 11. Precision-Recall curve for Stage 1 LinearSVC classifier

Utilizing these settings, the final model weighted F1-score for stage 1 was 82.10%, with class-wise performance of FR/NFR being 83.0% and 81.2%, respectively, which is balanced enough to support reliable first-pass routing. Figure 11 further shows that the classifier performs strongly across most of the recall range, with an average precision of 0.923, which indicates that the model was able to distinguish relevant requirements from irrelevant ones with high reliability. A similar study by Dev (2022) achieved an F1-score of 0.87 using a similar approach with SVM, which shows that the model is performing at a level that is consistent with related work.

8.2 Stage 2: The Specialist and Ablation Study

In stage 2, an ablation study was conducted to compare two approaches for non-functional multi-class classification: a class-weighted TF-IDF + LinearSVC baseline, and DistilRoBERTa. To ensure fairness between the comparison, both pipeline were tuned on the exact same training fold using stratified validation. For TF-IDF + LinearSVC, gridsearch was used again to optimize vectorizer settings. As such the best settings found were ngram range up to bi-grams, minimum document frequency of 3, and maximum document frequency of 0.85. The SVM regularization parameter C was then selected by 5-fold cross-validation with `class_weight='balanced'`. For DistilRoBERTa, the same split was used for a small hyperparameter sweep using AdamW, linear warmup and early stopping on validation loss (learning rates at $1e-5$ - $5e-5$, batch sizes 8-32 and epochs 2-5). Finally preprocessing was kept consistent where possible, such as lowercasing, punctuation, etc. As for the transformer, the model tokenizer was used with a fixed `max_length` to ensure consistency.

With both settings optimized, the result show that DistilRoBERTa achieved an F1-score of 80.08%, which is a significant improvement over the 74.71% F1-score of the linear SVC baseline.

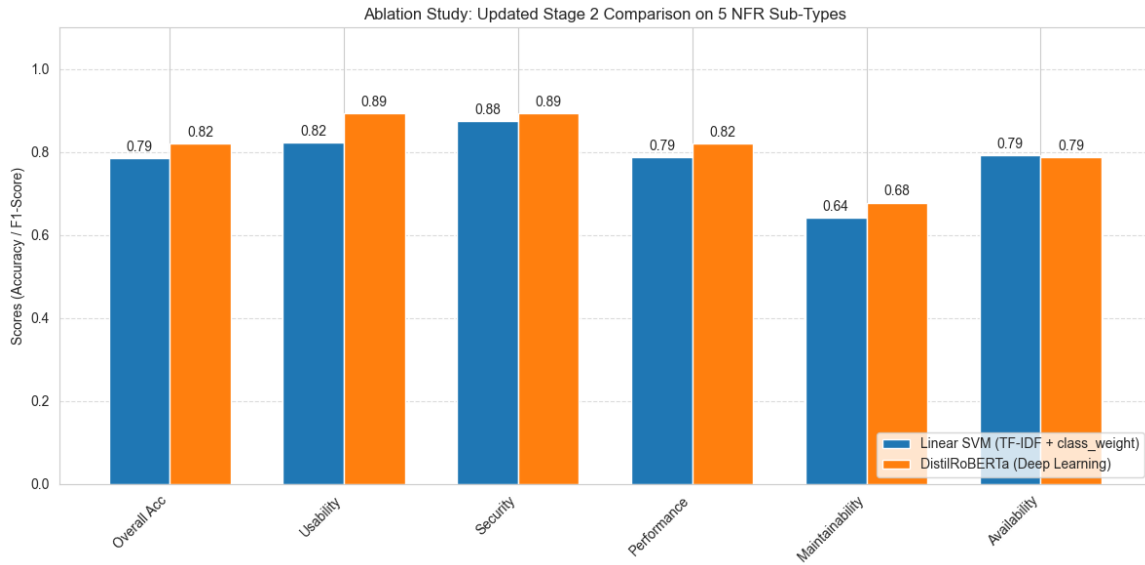


Fig. 12. Ablation study on Stage 2: optimized LinearSVC vs fine-tuned DistilRoBERTa

Figure 12 shows an ablation study comparing both models overall accuracy. The results show DistilRoBERTa outperformed the linearSVC baseline by 3.45% overall. Per class F1-scores show that the biggest gaps come from Performance and Usability, where DistilRoBERTa better handled paraphrases and contextual cues. Security classification had a modest gap (1%) but with the highest overall class performance, partly due to high domain-specific tokens such as “password” and “secure” driving decisions in both models.

Availability remained near-tied between models, which is notable because it has the smallest support in the dataset. This suggests that for very small classes, transformer gains may be limited, while for larger classes with more linguistic variety, the transformer specialist provides clearer benefit. Finally, Maintainability remained the weakest class with only a small gain, likely due to semantic overlap with other categories.

8.3 Stage 3: The Critic

The stage 3 classifier followed the same setup and ablation study as stage 2, but with a focus on binary ambiguity detection rather than subtype classification. As such, the same train-test split was used, and the same TF-IDF + LinearSVC pipeline was compared with a fine-tuned DistilRoBERTa model to see if the extra contextual understanding improves ambiguity detection.

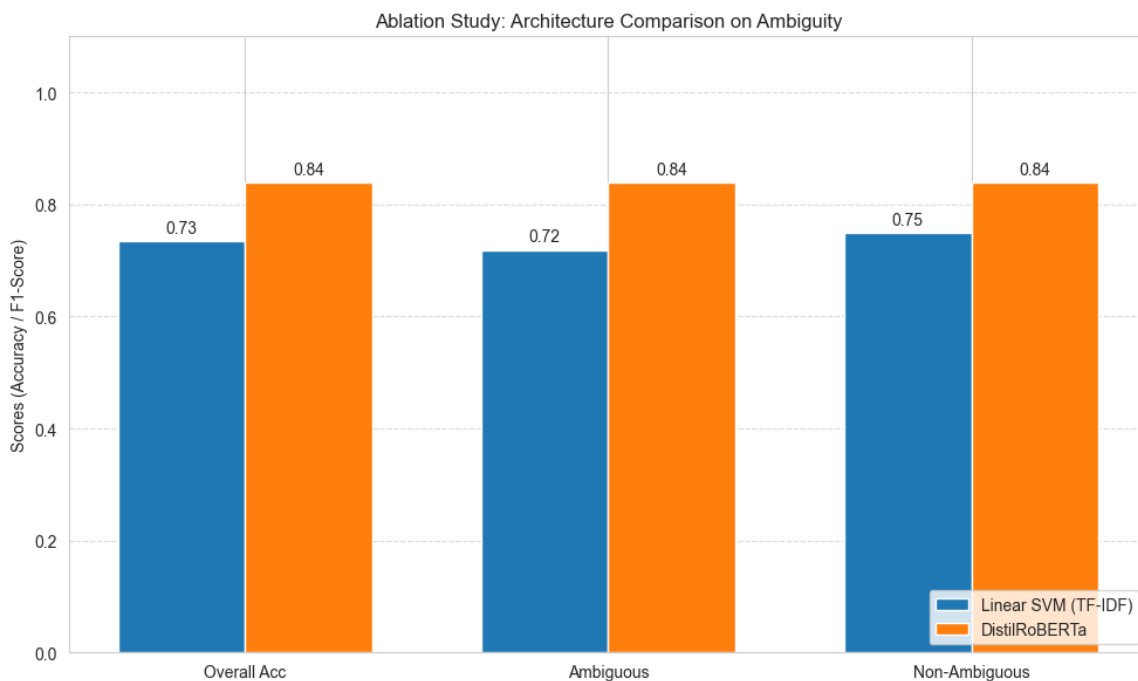


Fig. 13. Ablation study on Stage 3: LinearSVC vs DistilRoBERTa for ambiguity classification

Figure 13 shows another ablation study comparing both models overall accuracies. The results show that DistilRoBERTa outperformed the linearSVC baseline by 10.43% overall. Per class F1-scores show the largest improvement on the Ambiguous label (+12 percentage points), indicating the transformer better captures contextual and syntactic signals used to judge vagueness and therefore provides a more reliable selective trigger for the rewrite stage. The Non-Ambiguous label also improved but with a smaller gap, which is consistent with the idea that ambiguity detection relies heavily on specific linguistic cues that may be better captured by a transformer’s contextual understanding.

8.4 Stage 4: The Rewriter and Prompt Ablation

Stage 4 evaluates whether controller prompting with context injection can reduce ambiguity in requirements that were flagged by the Stage 3 classifier. As such, the rewrite model used is a local Llama 3.1 8B inference and compared four prompt variants under the same candidate set and decoding controls: plain, ambiguity_only, metadata, and metadata_rules. To keep the comparison fair, each variant rewrote the same inputs and was evaluated using the same automatic proxy, where rewritten outputs were passed back through the Stage 3 ambiguity classifier, and the post-rewrite predicted ambiguity rate was measured. Because the candidate pool was restricted to requirements already predicted as ambiguous, the baseline pre-rewrite ambiguity rate was 1.00.

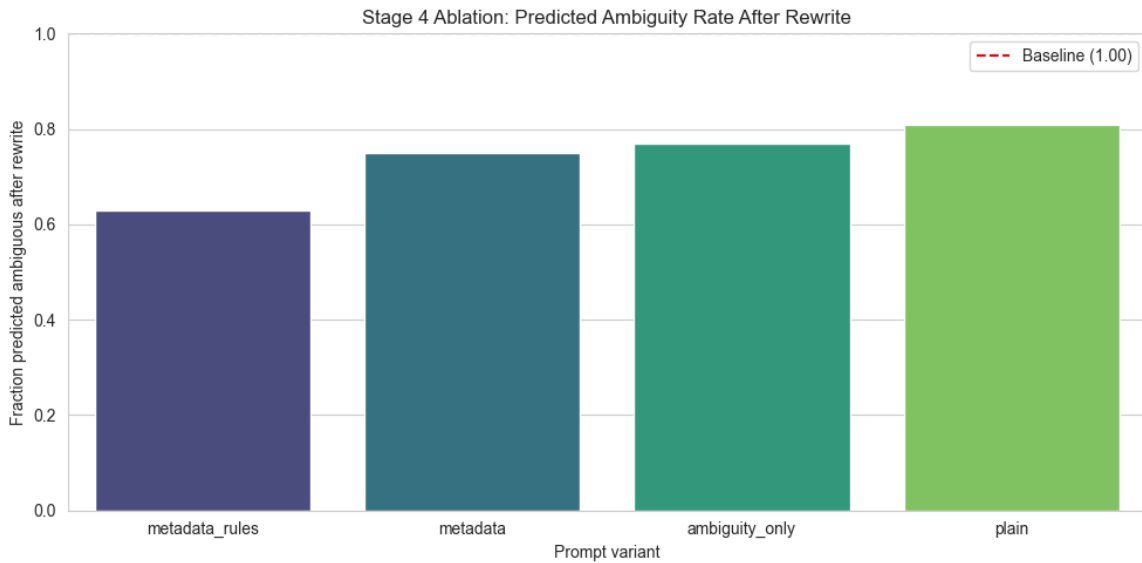


Fig. 14. Ablation study on Stage 4: Prompt variants for ambiguity reduction (lower is better)

The results showed a trend of consistent improvements as we introduced more metadata and structure into the prompts. According to figure 14, the plain prompt reduced the predicted ambiguity rate to about 0.81 (19% point reduction), while ambiguity_only reduced it further to about 0.77 (23% point reduction). Injecting upstream context produced additional gains: metadata reached about 0.75 (25% point reduction), and metadata_rules achieved the best result at about 0.63 (37% point reduction).

Table 11. Stage 4 rewrite example from notebook ablation outputs

Variant	Requirement text
Original input	A new stylistic map must have the right dimensions.
plain	The new stylistic map must have a predefined set of dimensions that match the product's design specifications.
ambiguity_only	The new stylistic map must have a predefined set of dimensions that match the product's design specifications.
metadata	The new stylistic map must have a fixed aspect ratio and be designed to fit within the standard display area of the application, ensuring optimal viewing and interaction for users.
metadata_rules	The new stylistic map must have a fixed aspect ratio and be designed to fit within the available screen real estate, ensuring that users can easily navigate and interact with it without unnecessary scrolling or zooming.

Table 11 shows an example of the same requirement rewritten by the different prompt variations. The original requirement is flagged ambiguous due to the vague phrase ‘right dimensions’, which does not define how acceptability is judged. Both the `plain` and `ambiguity_only` rewrites improved the wording but remained relatively generic through terms such as ‘predefined’ and ‘design specifications’. Once the metadata is introduced into the prompt, the `metadata` rewrite becomes more understandable by introducing interface-related constraints such as ‘fixed aspect ratio’ and ‘display-area fit’, which are more specific and quantifiable.

9 Discussion

Synthesizing the results of all 4 stages, we can assume that the pipeline was able to meet the core aim of this dissertation. However, it is important to discuss the limitations and trade-offs of each stage, as well as the rationale behind the design choices.

For stage 1, the TF-IDF + LinearSVC classifier was kept as the first stage as it was able to give a reliable first pass, with 82.10% accuracy while also maintaining low cost (Figure 11). Other reasons to choose a classic ML model over a more complex transformer lies in speed and reproducibility. Using a heavier model here would increase computing costs for a stage that mainly filters obvious functional vs non-functional cases. What did not work well however was deeper semantic separation: some requirements with overlapping words were still difficult to route cleanly, and stronger text normalization can also remove useful nuance, which a transformer might have handled better. Even with those limits, this setup was chosen as the first phase filter.

For stage 2, the comparison between the classic LinearSVC vs a transformer specialist (DistilRoBERTa) showed that the transformer improved overall performance over the class-weighted linear baseline (81.99% vs 78.54%; Figure 12). This marginal increase in performance in multiclass classification can be stemmed from the transformer’s ability to understand context and handle paraphrasing. Especially in cases where the same requirement could be expressed in different ways. That said, not all NFR subclasses benefitted equally. ‘Maintainability’ remained comparatively weak for both models. A confusion matrix analysis revealed that maintainability requirements were often misclassified as usability or performance the most, which suggest there might be more semantic overlap between these categories, a future improvement could be to add more dataset entries to improve the model’s ability to learn the subtle differences. Finally, smaller support labels such as ‘Availability’ showed less consistent gains, which is likely due to the limited number of examples for the model to learn from, even after applying SMOTE. We still selected DistilRoBERTa here because this stage is exactly where semantic context matters most, even if the gains are uneven across labels.

For stage 3, we compared the same linear baseline with a transformer specialist, but with a different preprocessing protocol. Stop words and modal cues, such as ‘might’, ‘could’, etc. were intentionally retained in the training set. This is because ambiguity detection relied on these signals, and removing them would have reduced the model’s ability to learn from these cues (Berry et al. 2003b). Eventually the choice worked comparatively well, as the LinearSVC baseline reached an F1-score of 73.48%, while DistilRoBERTa reached 83.91% and improved the Ambiguous class performance by 12 percentage points (Figure 13). What did not work well was calibration on borderline sentences. As some requirements are difficult even for humans to label, false positives and false negatives still occurred around vague but context-dependent wording. A way to mitigate this issue in the future could be to add a third ‘borderline’ class for requirements that are difficult to label, and then use a separate model or rule based system to handle these cases.

Finally for stage 4, the ablation study showed that the prompting strategy of adding metadata and subtype-aware guidance did in fact improve the LLM's ability to reduce ambiguity. While all variants reduced predicted ambiguity from baseline, the stronger gains came only after adding metadata and subtype-aware guidance (metadata_rules from 1.00 to about 0.63, versus plain to about 0.81; Figure 14). This goes to show that previous stages worked hand in hand to provide useful metadata signals for the rewrite model. However, a -36% point reduction still left a significant portion of requirements predicted as ambiguous, which suggests while the aim of the pipeline is in the right direction, there is still room for improvement to introduce stronger constraints on the rewrite model. Furthermore, the evaluation protocol itself is limited by the fact that it relies on the same ambiguity classifier to judge the rewritten outputs, which may not perfectly align with human judgments of ambiguity. As such, we suggest a future evaluation study could involve human annotators assessing the rewritten requirements for clarity and its ability to meet the stakeholders needs.

10 Conclusion

This dissertation began by highlighting the persistent crisis in traditional requirements engineering. As poorly defined software requirements remain a leading cause of project failure, often resulting in operational collapses such as the Heathrow Terminal 5 disaster (Milmo 2008). As such, the core aim of this dissertation was to propose an automated framework that combines traditional machine learning and modern language models to enhance the requirements analysis process, while balancing speed with linguistic depth.

The experimental analysis result (section 8) shows that this architecture is rather effective. Stage 1 (TF-IDF+SVM) delivered solid binary gatekeeping performance for functional/non-functional routing, confirming that classical ML remains highly competitive for structured first pass classification. The decision to add a Stage 2 DistilRoBERTa specialist instead of relying purely on linear SVM stemmed from its stronger semantic modeling on 5-class subtype boundaries, as demonstrated by the ablation gap against the optimized SVM baseline. Stage 3 further reinforced this pattern by improving ambiguity classification under the revised protocol, enabling reliable passthrough to the generative stage. Finally, Stage 4 was able to show that Llama 3.1 can rewrite ambiguous requirements into clear and testable sentences when guided by structured prompts and upstream signals, albeit with some limitations.

Taken together, these findings do support the dissertation's main claim, that the future of Requirements Engineering is not about replacing Machine Learning with Large Language Models, but strategically chaining them together. Rather than positioning models as competitors, the pipeline treats them as complementary components in a coherent structure. Classical ML provides speed and stable routing, transformer specialists provide semantic discrimination, and generative LLMs provide quality enhancement where ambiguity is detected.

10.1 Research Contributions

This dissertation contributes to three key areas. First, we contribute a highly curated dataset of 4000 requirements extracted from public archives (Ferrari et al. 2017), (corpus4panwo 2023), (pro 2026), with detailed annotations for functional/non-functional classification, NFR subtypes, and ambiguity labels. This dataset can be acted as a valuable resource for future research in automated requirements analysis which can also be used to benchmark new models and approaches. Secondly, a hybrid pipeline architecture was proposed and implemented, which was able to combine classical machine learning and modern language models to address different aspects of requirements analysis. This architecture demonstrates how different model families can be orchestrated in a complementary way to achieve better overall performance than any single model could provide. Lastly, the dissertation provides empirical evidence through ablation studies that justify the design choices at each stage, showing where classical methods are sufficient and where transformer-based models provide necessary improvements. This evidence supports the rationale for a hybrid

approach and offers insights into the strengths and limitations of each model type in the context of requirements engineering.

10.2 Limitations

Despite the strong results of the proposed pipeline, several limitations should be acknowledged when interpreting the findings and planning future work. First, the evaluation of the model is bounded by the annotation assumptions of the dataset. This coverage and balance of the dataset may not fully represent the diversity of real world requirements. As such, the performance of the pipeline on unseen domains or different types of requirements may vary. An improvement on this could be to expand the dataset with more domain specific examples.

Secondly, the stage 4 rewrite evaluation relies on the same ambiguity classifier to judge the rewritten outputs, which may not perfectly align with human judgments of ambiguity. This means that while the model may reduce the predicted ambiguity rate according to the classifier, it may not necessarily produce requirements that are clearer or more testable from a human perspective. Future studies could use 2–3 human annotators with cross-validation to better assess the quality of the rewritten requirements, especially in terms of clarity and alignment with stakeholder needs.

10.3 Scope for Future Work

Future work could focus on improving the pipeline based on the limitations discussed in the previous section.

- (1) **Dataset expansion and domain transfer:** evaluate the pipeline across additional industrial domains and multilingual requirement sets.
- (2) **Calibration and uncertainty:** add confidence-aware routing and rejection strategies to improve robustness in borderline cases.
- (3) **Rewrite evaluation:** extend beyond lexical similarity metrics with stronger human-grounded quality audits aligned to ISO/IEC/IEEE 29148 criteria.
- (4) **Operational deployment:** integrate reviewer feedback loops and active-learning updates so the system continuously improves from human corrections.

In conclusion, this dissertation demonstrates that hybrid orchestration is a practical and high-performing direction for automating Requirements Analysis. By combining the reliability of classical ML with the semantic and generative strengths of modern language models, the proposed framework advances both the technical state of practice and the methodological direction of future research in Requirements Engineering.

References

2026. <https://www.kaggle.com/datasets/iamsouvik/promise>
- Khlood Ahmad, Muneera Bano, Mohamed Abdelrazek, Chetan Arora, and John Grundy. 2021. What's Up with Requirements Engineering for Artificial Intelligence Systems?. In *2021 IEEE 29th International Requirements Engineering Conference (RE)*. 1–12. doi:10.1109/RE51729.2021.00008
- Kolade Ajeigbe and Mark Maddog. 2021. Natural Language Processing for Requirement Extraction. (08 2021).
- Muneera Bano. 2015. Addressing the challenges of requirements ambiguity: A review of empirical literature. In *2015 IEEE Fifth International Workshop on Empirical Requirements Engineering (EmpiRE)*. 21–24. doi:10.1109/EmpiRE.2015.7431303
- Lodiana Beqiri, Calkin Suero Montero, Antonio Cicchetti, and Andrey Kruglyak. 2024. Classifying Ambiguous Requirements: An Explainable Approach in Railway Industry. In *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*. 12–21. doi:10.1109/REW61692.2024.00007
- Daniel Berry, Erik Kamsties, Michael Krieger, J Mathematics, Mkrieger239, D Berry, E Kamsties, and M Krieger. 2003a. *From Contract Drafting to Software Specification: Linguistic Sources of Ambiguity A Handbook Version 1.0*. <https://cs.uwaterloo.ca/~dberry/handbook/ambiguityHandbook.pdf>
- Daniel Berry, Erik Kamsties, Michael Krieger, J Mathematics, Mkrieger239, D Berry, E Kamsties, and M Krieger. 2003b. *From Contract Drafting to Software Specification: Linguistic Sources of Ambiguity A Handbook Version 1.0*. <https://cs.uwaterloo.ca/~dberry/handbook/ambiguityHandbook.pdf>
- Rajendra Budake, Sudhakar Bhoite, and Kabir Kharade. 2023. A Study of AI-Based Techniques for Requirement Analysis in Software Engineering. In *AIP Conference Proceedings*. 050007. doi:10.1063/5.0178114
- Agustin Casamayor, Daniela Godoy, and Marcelo Campo. 2009. Semi-Supervised Classification of Non-Functional Requirements: An Empirical Analysis. *Inteligencia Artificial* (May 2009). doi:10.4114/ia.v13i44.1044
- Haowei Cheng, Jati H. Husen, Yijun Lu, Teeradaj Racharak, Nobukazu Yoshioka, Naoyasu Ubayashi, and Hironori Washizaki. 2024. Generative AI for Requirements Engineering: A Systematic Literature Review. *arXiv preprint arXiv:2409.06741* (2024).
- corpus4panwo. 2023. Fault-prone SRS Dataset. <https://www.kaggle.com/datasets/corpus4panwo/fault-prone-srs-dataset>
- Jayant Dave Dev. 2022. *Identifying Functional and Non-Functional Software Requirements from User App Reviews and Requirements Artifacts*. <https://digitalcommons.montclair.edu/cgi/viewcontent.cgi?article=2014&context=etd>
- Alessio Ferrari, Giorgio Oronzo Spagnolo, and Stefania Gnesi. 2017. PURE: A Dataset of Public Requirements Documents. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*. 502–505. doi:10.1109/RE.2017.29
- Assem Hawari. 2019. A Dataset of Mobile Application Reviews for Classifying Reviews into Software Engineering Maintenance Tasks Using Data Mining Techniques. doi:10.17632/5fk732vkw.2
- Cory Hymel and Hiroe Johnson. 2025. Analysis of LLMs vs Human Experts in Requirements Engineering. *arXiv preprint arXiv:2501.19297* (2025).
- ISO. 2018. ISO 31000:2018 Risk management — Guidelines. <https://www.iso.org/standard/65694.html>
- Rahat Izhar, Shahid N. Bhatti, and Sultan A. Alharthi. 2025. Bridging Precision and Complexity: A Novel Machine Learning Approach for Ambiguity Detection in Software Requirements. *IEEE Access* 13 (2025), 12014–12031. doi:10.1109/ACCESS.2025.3529943
- Fredrik Karlsson, Panagiota Chatzipetrou, Shang Gao, and Tanja Elina Havstorm. 2025. How Reliable Are GPT-4o and LLAMA3.3-70B in Classifying Natural Language Requirements? The Impact of the Temperature Setting. *IEEE Software* 42, 6 (2025), 97–104. doi:10.1109/MS.2025.3572561
- Ravleen Kaur and Mohinder :Pal Singh Bhatia. 2025. *Managing Uncertainty in NLP: Advanced Techniques and Approaches*. 101–130. doi:10.1007/978-3-031-88988-2_5
- Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. 2019. CTRL: A Conditional Transformer Language Model for Controllable Generation. *CoRR* abs/1909.05858 (2019).

- arXiv:1909.05858 <http://arxiv.org/abs/1909.05858>
- Hyuhng Joon Kim, Youna Kim, Cheonbok Park, Junyeob Kim, Choonghyun Park, Kang Min Yoo, Sang-goo Lee, and Taeuk Kim. 2024. Aligning Language Models to Explicitly Handle Ambiguity. *arXiv preprint arXiv:2404.11972* (2024).
- Elizabeth Larson. 2014. I Still Don't Have Time to Manage Requirements: My Project Is Later Than Ever. <https://www.pmi.org/learning/library/poor-requirements-management-source-failed-projects-9341>
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2021. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. arXiv:2107.13586 [cs.CL] <https://arxiv.org/abs/2107.13586>
- Mengmeng Lu and Peng Liang. 2017. Automatic Classification of Non-Functional Requirements from Augmented App User Reviews. doi:10.1145/3084226.3084241
- Aaron K. Massey, Richard L. Rutledge, Annie I. Antón, and Peter P. Swire. 2014. Identifying and Classifying Ambiguity for Regulatory Requirements. In *2014 IEEE 22nd International Requirements Engineering Conference (RE)*. 83–92. doi:10.1109/RE.2014.6912250
- Dan Milmo. 2008. Passengers Fume in the Chaos of Terminal 5's First Day. <https://www.theguardian.com/environment/2008/mar/28/travelandtransport.theairlineindustry>
- Project Management Institute (PMI). 2014. *Requirements Management: A Core Competency for Project and Program Success (2014 In-Depth Report)*. <https://www.pmi.org/-/media/pmi/documents/public/pdf/learning/thought-leadership/pulse/requirements-management.pdf>
- PYTHON. 2025. Python. <https://www.python.org/>
- António Miguel Rosado da Cruz and Estrela Ferreira Cruz. 2025. Machine Learning Techniques for Requirements Engineering: A Comprehensive Literature Review. *Software* 4, 3 (2025). doi:10.3390/software4030014
- António M. Rosado da Cruz and Estrela Ferreira Cruz. 2025. Artificial Intelligence Techniques for Requirements Engineering: A Comprehensive Literature Review. *Preprints* (Mar 2025). doi:10.20944/preprints202503.2259.v1
- Scikit-learn. 2024. scikit-learn: Machine Learning in Python. <https://scikit-learn.org/stable/>
- Md Mehrab Tanjim, Xiang Chen, Victor S. Bursztny, Uttaran Bhattacharya, Tung Mai, Vaishnavi Muppala, Akash Maharaj, Saayan Mitra, Eunye Koh, Yunyao Li, et al. 2025. Detecting Ambiguities to Guide Query Rewrite for Robust Conversations in Enterprise AI Assistants. *arXiv preprint arXiv:2502.00537* (2025).
- Archana Tikayat Ray, Bjorn F. Cole, Olivia J. Pinon Fischer, Ryan T. White, and Dimitri N. Mavris. 2023. aeroBERT-Classifier: Classification of Aerospace Requirements Using BERT. *Aerospace* 10, 3 (2023). doi:10.3390/aerospace10030279
- Xinggang Wang, Buyun Sheng, Liang Xue, and Zheng Xiao. 2016. Classification of Customer Requirements on MapReduce-Based Naive Bayes. In *2016 IEEE International Conference on Big Data Analysis (ICBDA)*. 1–4. doi:10.1109/ICBDA.2016.7509803
- Mohammad Amin Zadenoori, Jacek Dąbrowski, Waad Alhoshan, Liping Zhao, and Alessio Ferrari. 2025. Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. *arXiv preprint arXiv:2509.11446* (2025).
- Fatima Zait and Nacereddine Zarour. 2018. Addressing Lexical and Semantic Ambiguity in Natural Language Requirements. In *2018 Fifth International Symposium on Innovation in Information and Communication Technology (ISIICT)*. 1–7. doi:10.1109/ISIICT.2018.8613726
- Kareshna Zamani, Didar Zowghi, and Chetan Arora. 2021. Machine Learning in Requirements Engineering: A Mapping Study. In *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)*. 116–125. doi:10.1109/REW53955.2021.00023
- Yizhuo Zhang and Yanhui Li. 2024. The Classification of Software Requirements using Large Language Models: Emerging Results and Future Directions. In *2024 3rd International Conference on Cloud Computing, Big Data Application and Software Engineering (CBASE)*. 746–749. doi:10.1109/CBASE64041.2024.10824444

A Project Management

The project management will consist of several key components to ensure the successful delivery of the research objectives. In this section, we will outline the project's scope, followed by the definition of its deliverables, then finally the project's plan. In the next section we will outline the project's timeline and milestones in the form of a Gantt chart. Following this, we will identify potential risks that may impact the project and outline mitigation strategies to address them. Finally, we will outline the project's standing in the PLES framework, identifying its position in terms of Professional, Legal, Ethics, and Social impact.

A.1 Project Scope

The scope of this project is to develop a hybrid framework that leverages both classical machine learning and generative AI to enhance the requirements engineering process. We will be focusing on two main tasks: (1) the classification of requirements into functional and non-functional categories, and (2) detection and resolution of ambiguity within these requirements. This will be achieved in a 2-step process, where an SVM classifier will first detect ambiguous keywords in requirements, then a Llama 3.1 model will be used to rewrite these ambiguous requirements into clear, specific, and verifiable statements. The project will converge into a prototype tool that integrates these components and will be evaluated against a manually annotated "ground truth" dataset.

A.2 Project Deliverables & Plan

This project will be divided into 4 deliverables, each aligning with the specified deadline of this course.

A.2.1 Deliverable 1 & 2. The first two deliverables, both due in week 11 of semester 1, focused on the initial idea and research gap found, validated by a comprehensive literature review, then followed by the planning of the methodology and drafting an ethics approval form reviewed by the ethics committee. Finally, a feasibility study will be conducted and presented via a pilot study and a mini-video presentation to ensure the practicality of the proposed methodology.

A.2.2 Deliverable 3. The third deliverable, due in week 11 of semester 2, will be focused on the implementation of the prototype tool. This includes the development of the SVM classifier for requirement classification and ambiguity detection, as well as the integration of the Llama 3.1 model for ambiguity resolution. The prototype's performance will be benchmarked using Precision, Recall, F1-score, Accuracy, and rewrite-stage ablation indicators to validate that the hybrid SVM-LLM approach provides a quantifiable improvement over using a large language model in isolation.

A.2.3 Deliverable 4. Throughout the module, the final deliverable required a logbook of the work carried out. Accordingly, records will be tracked and shared with the appropriate

supervisor via: Github for code versioning, OneDrive for document sharing, and weekly progress reports via email and meetings.

A.3 Gantt Chart

The Gantt chart for the project is delivered in two sections. Each section is carried out over a period of 1 semester (12 weeks). The first semester focuses on background research and data processing, while the second semester will focus on model development, evaluation, and final dissertation writing.

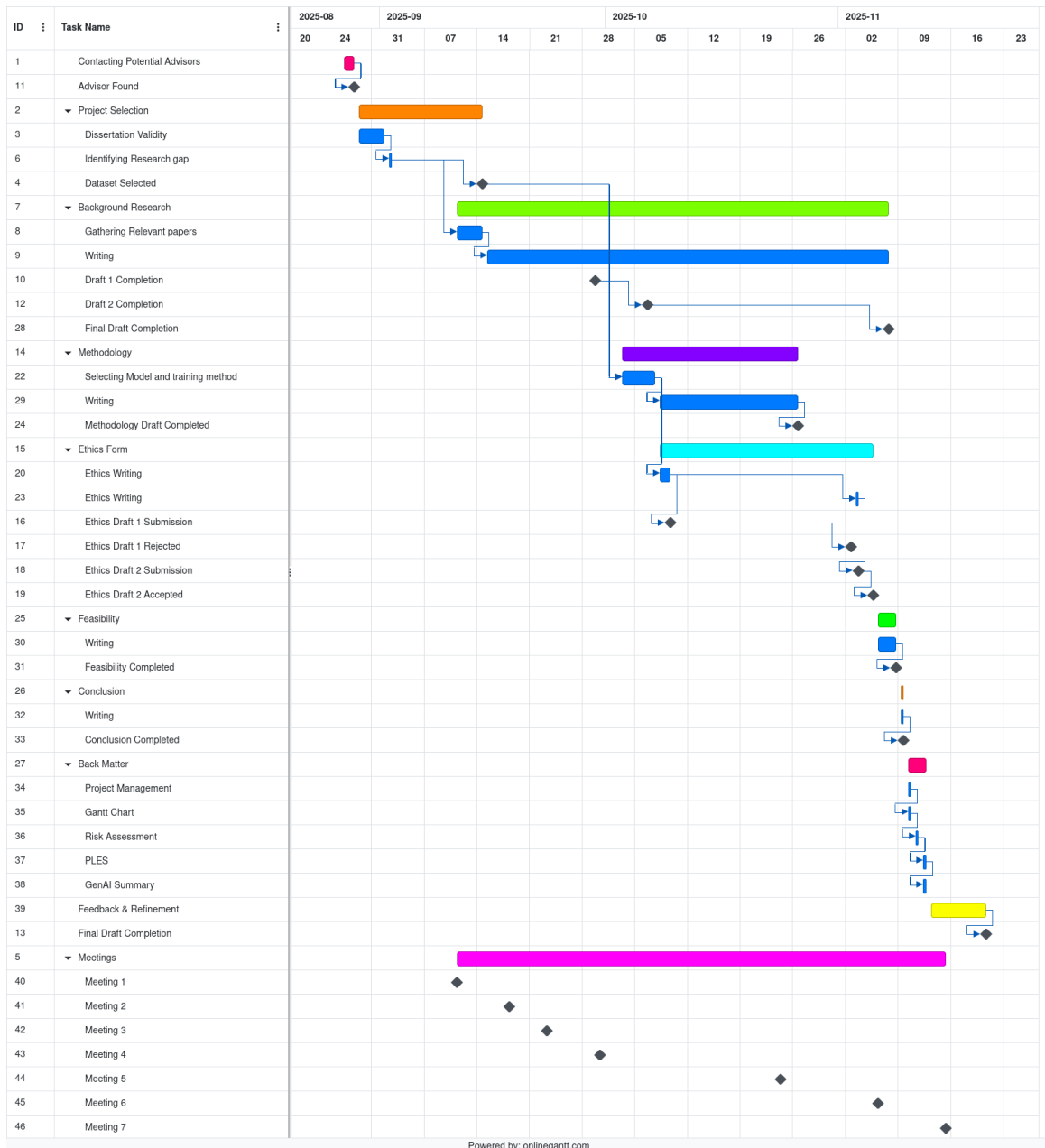


Fig. 15. Semester 1 Gantt Chart

Accelerating Requirements Analysis: Machine Learning & LLMs for Automated Classification and Quality Checks to Improve Software Specification

• 39



Fig. 16. Semester 2 Gantt Chart

A.4 Risk Management

To ensure that the project meets its deadlines and objectives, a risk assessment is crucial to identify and mitigate these potential challenges. Therefore, this dissertation's risk management process will be adapted from the ISO 31000:2018 guidelines (ISO 2018), ensuring that the outlined risks are quantifiable and systematically addressed. The following section will identify the risk factors as followed:

- **Risk Identification:** Identifying the types of risks that could potentially impact the project timeline, experiment, or deliverables.
- **Risk Analysis:** Analysis of each risk by the Likelihood (L) and impact (I) caused. This will be on a scale of 1-5, with 1 being the lowest and 5 being the highest.
- **Risk Prioritization:** Calculated via the Risk Priority Number (RPN) = $L \times I$, this will help prioritize which risks need to be addressed first.
- **Risk Mitigation Strategies:** For each identified risk, a mitigation strategy will be developed to minimize its impact or likelihood.

A.5 Risk Prioritization Matrix

The table below outlines the risk prioritization matrix, which will be using to calculate the RPN for each identified risk:

Scale	Likelihood (L)	Impact (I)
High (5)	Very Likely	Potential Dissertation Failure
Med-High (4)	Likely	Major grade reduction penalty
Med (3)	Possible	Significant Rework Required
Med-Low (2)	Unlikely	Minor delays & Small Rework
Low (1)	Very Unlikely	Minimal Impact

Table 12. Risk Prioritization Matrix

A.6 Risk Priority Number Threshold

The following table outlines the RPN thresholds that will be used to prioritize the risks in the next section:

RPN Range	Priority Level
15-25	High Priority - Immediate Action Required
8-14	Medium Priority - Monitor and Mitigate
1-7	Low Priority - Acceptable Risk

Table 13. Risk Priority Number (RPN) Thresholds

A.7 Risk Management Table

Combining the identified risks with the prioritization framework, then identifying the threshold, we can now create a prioritized risk management table, as seen below:

ID	Risk	L	I	RPN	Mitigation Strategy
1	"Ground Truth" Annotation bottleneck	5	3	15	Start with a smaller, high-quality batch (150-300 items) to build a "v1" ground truth. Prioritize manual accuracy over quantity.
2	Poor Time Management	3	4	12	Develop a detailed project plan; set regular milestones and reviews
3	Software Bugs and Integration Issues	3	4	12	Regular code reviews and testing; modular development approach
4	Classifier Failure (Class Imbalance)	3	4	12	Use techniques like SMOTE to balance classes; Train the SVM using the class_weight='balanced' parameter. Evaluate using the F1-Score, not accuracy.
5	Ineffective Feature Engineering (TF-IDF)	3	4	12	If TF-IDF gives a low F1-score, the backup plan is to switch to pre-trained sentence-embeddings (e.g., SBERT) for vectorization.

Table 14. Risk Management Table

ID	Risk	L	I	RPN	Mitigation Strategy
6	Model Performance Below Expectations	2	5	10	Implement iterative testing and tuning; explore alternative methods if necessary
7	Non-Reproducible LLM Results	2	5	10	Set the LLM's temperature parameter to 0 or 0.1 to force deterministic (reproducible) outputs for evaluation.
8	Contextual blindness in LLM outputs	2	5	10	Use few-shot learning with contextual examples in the prompt to guide the LLM towards domain-specific interpretation
9	Data Quality Issues	3	3	9	Conduct thorough data cleaning and validation; use multiple data sources
10	Computational Resource Limitations	3	2	6	Utilize paid GPU services & optimize code for efficiency
11	Academic Misconduct	1	5	5	Ensure regular meetings with advisor to ensure progress is on the right track

Table 15. Risk Management Table (Continued)

A.8 PLES

The study in this dissertation follows the guidelines aforementioned by the university ensuring the compliance for any professional, legal, ethical, and societal considerations are met, as such the PLES framework is outlined in the subsections below:

A.8.1 Professional Considerations. The research ensures all work to be done professionally, adhering to the university's code of conduct. All chapters have been transparently documented and properly referenced with the relevant authors, ensuring that no plagiarism occurs. The project experiment also upholds its standard of integrity by ensuring that all results are reported honestly and are quantifiable, via the use of Precision, Recall and F1-score metrics. Finally, to ensure the experiment is reproducible, the tools discussed in the feasibility such as Python, Google Colab, Llama 3.1, etc. are all open-source and publicly accessible.

A.8.2 Legal Considerations. The project has been designed to fully comply with relevant data protection rules and intellectual property rights. All data used in this experiment has been properly sourced from the PURE dataset (Ferrari et al. 2017), which is a publicly available and open-source dataset. The data also ensures that no Personally Identifiable Information (PII) have been identified or used in the experiment. Finally, any software libraries and tools used in the project are open-source and properly licensed for academic use.

A.8.3 Ethical Considerations. The project adheres and is ethically approved by the university's ethics committee. As mentioned, no human participants are involved in the experiment. However, the black-box nature of LLMs can lead to ethical concerns, such as bias in outputs and inexplicability in decision-making processes. Therefore, it should be noted that this tool must only be used as an assistant to a human engineer (a "human-in-the-loop" model) and never as a fully automated final decision-maker.

A.8.4 Societal Considerations. The project's aim to "Accelerate Requirements Analysis" inadvertently has direct social implications. As mentioned in the ethical considerations, the tool is not designed to replace human engineers by to assist them in their tasks. This frees the human engineer to focus on high-value social tasks that an AI cannot, such as stakeholder negotiation, creative problem-solving, and understanding complex business context. It is also to be mentioned that the tool cannot be 100% trusted and may produce "hallucinations", that is producing an answer that is incorrect but sounds highly plausible. This furthers the reason the need for a "human-in-the-loop".

B Generative AI

The application of GenAI in this dissertation was used in an appropriate amount as per the university's guidelines. Which was used in the following ways:

- Brainstorming and developing initial research ideas.
- Find and summarize relevant existing literature for the background research chapter.
- Identify misspellings and provide grammar suggestions in writing.
- Provide code snippets for latex formatting.
- Identify potential tools needed for the feasibility study.
- Assist in code optimization for the project.

The following showcases what GenAI was NOT used for in this dissertation:

- The overall research idea, aim, and objectives.
- Being the sole writer of any chapter.
- Coding the entire project.

C Project Proposal and Research Changes

Due to some limitations and methodological adjustments, two significant changes were made to the implementation.

First, the scale of the experiment dataset was substantially expanded beyond the initial proposal. During early stages, we proposed a dataset of 500 entries minimum to train the models. However, after much discussion with the supervisor, it became apparent that a larger corpus was mathematically required to effectively utilize synthetic balancing techniques (e.g., SMOTE) and prevent the Stage 2 multi-class transformer from overfitting to minority classes. By utilizing our proposed risk assessment framework (ID 9), we were able to justify the expansion by incorporating additional data sources to the curation.

Second, the evaluation protocol for the Stage 4 generative LLM was fundamentally altered. The original proposal outlined the use of BERTScore to measure the semantic similarity of the LLM outputs against a manually curated “Golden Rewrite” (GR) set. The validity of this approach was suitable, but was introduced with a bottleneck of manual ground-truth curation, and cross-human validation, which was not possible to host without extra-careful planning. As such, a closed loop validation protocol was introduced, where the LLM’s rewritten outputs are fed directly back into the trained Stage 3 Ambiguity Critic. Limitations on this has been discussed in the conclusion section.

D Supplementary Material

This section contains additional charts generated from pipeline.ipynb that support the analysis but were not included in the main chapters.

D.1 Data Profiling

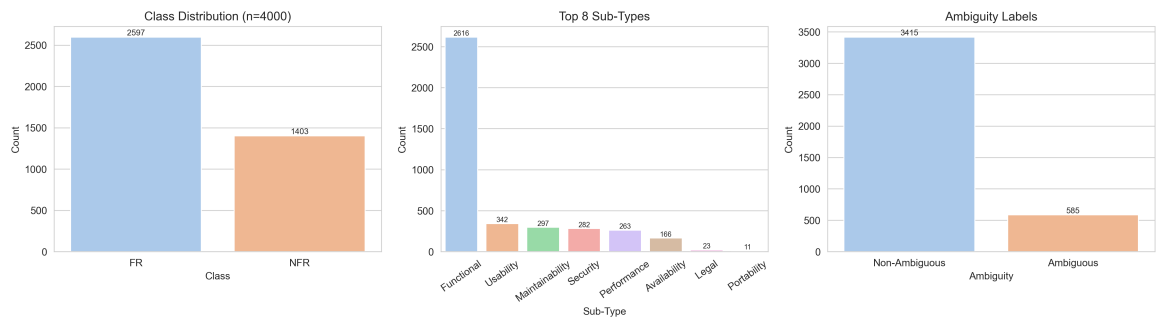


Fig. 17. Alternative dataset profiling view generated during pipeline data exploration

D.2 Stage 1 Supplementary Charts

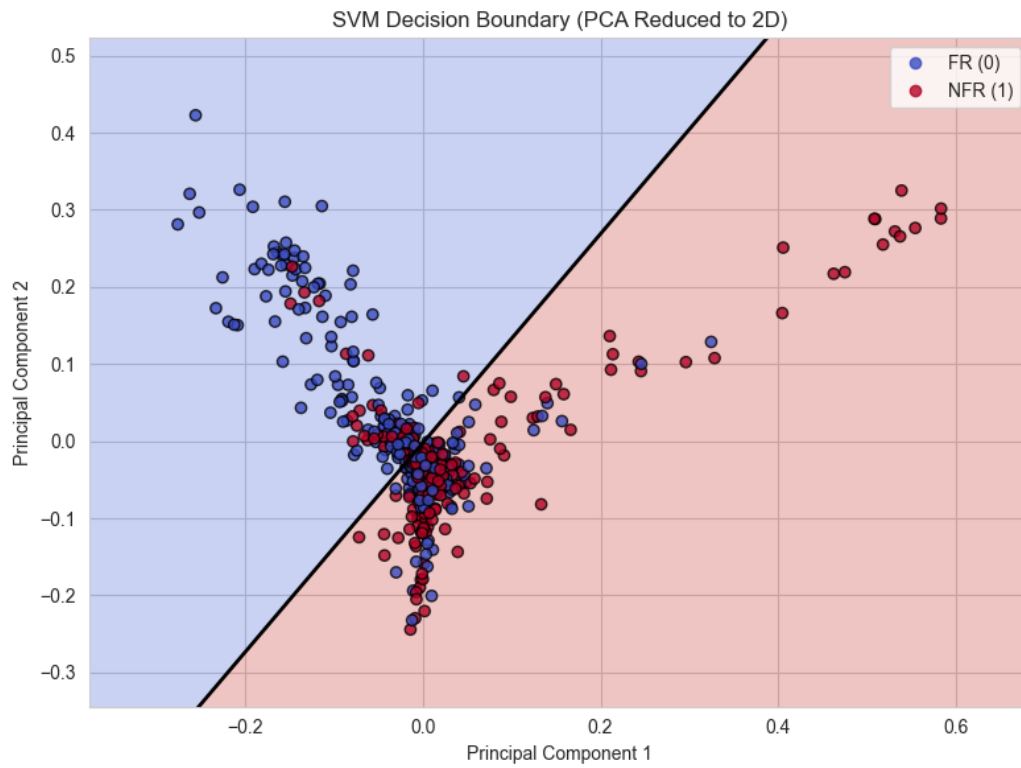


Fig. 18. Stage 1 LinearSVC confusion matrix (FR vs NFR) from notebook evaluation outputs

D.3 Stage 2 Supplementary Charts

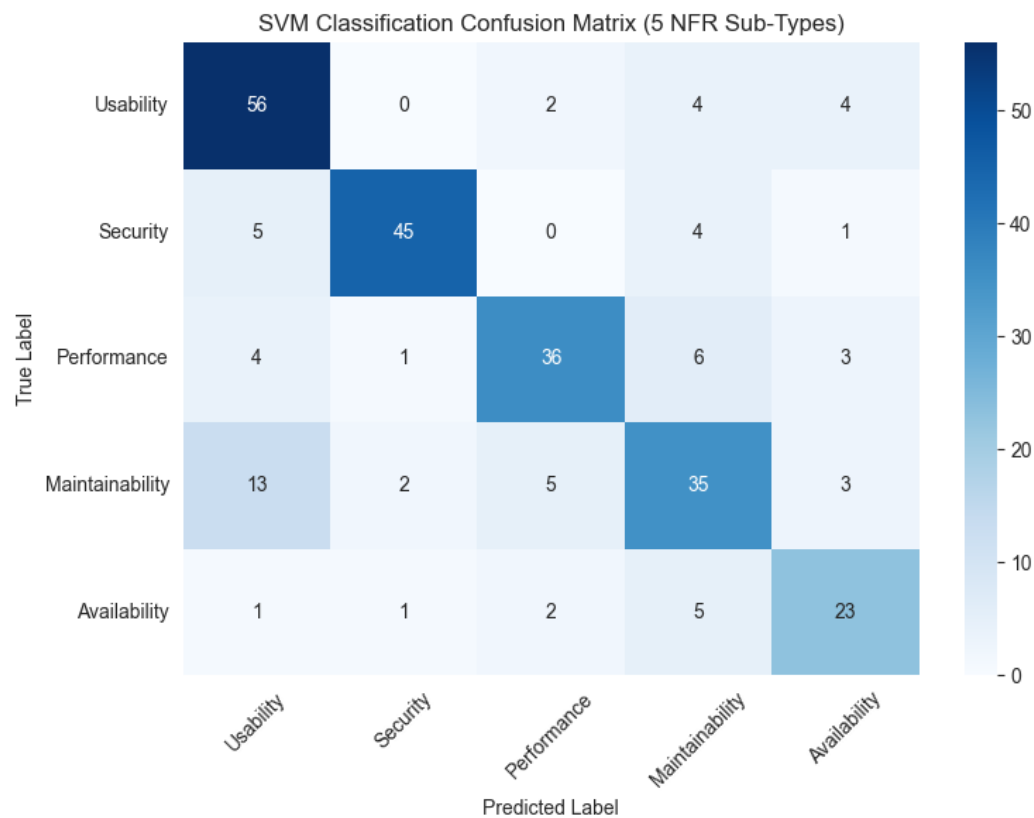


Fig. 19. Stage 2 optimized LinearSVC confusion matrix for 5-class NFR subtype classification

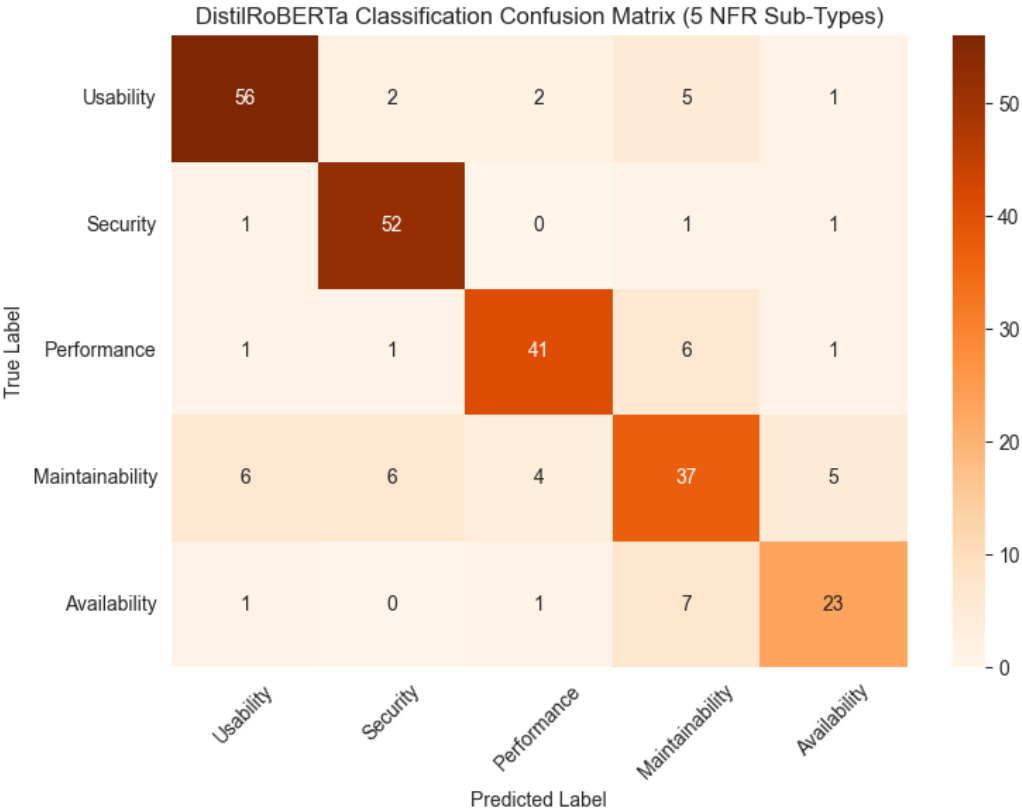


Fig. 20. Stage 2 DistilRoBERTa confusion matrix for 5-class NFR subtype classification

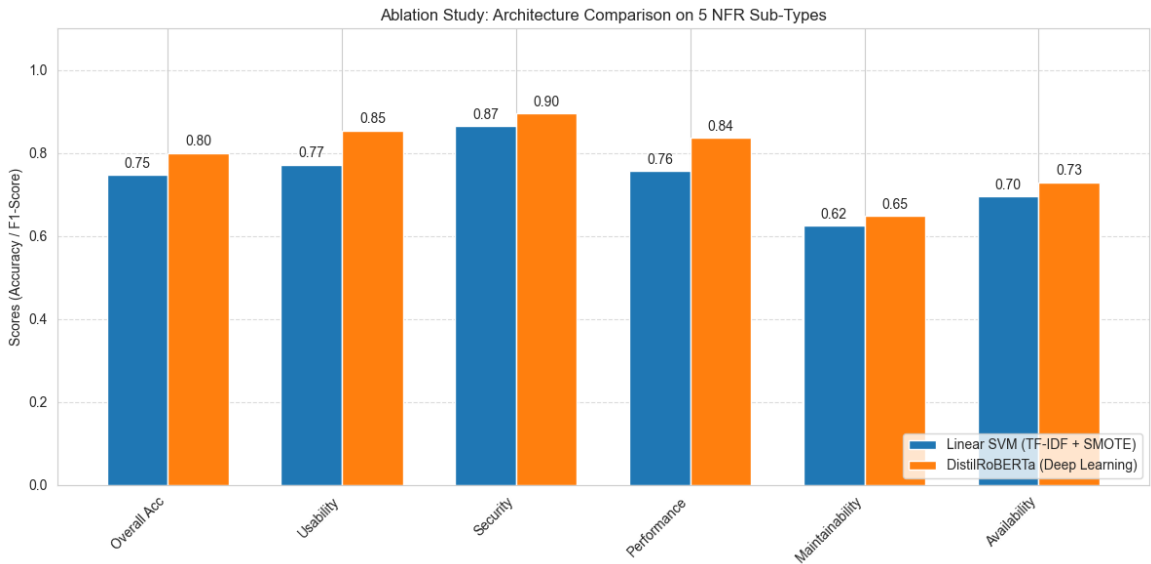


Fig. 21. Stage 2 ablation comparison chart (alternative notebook export)

D.4 Stage 3 Supplementary Charts

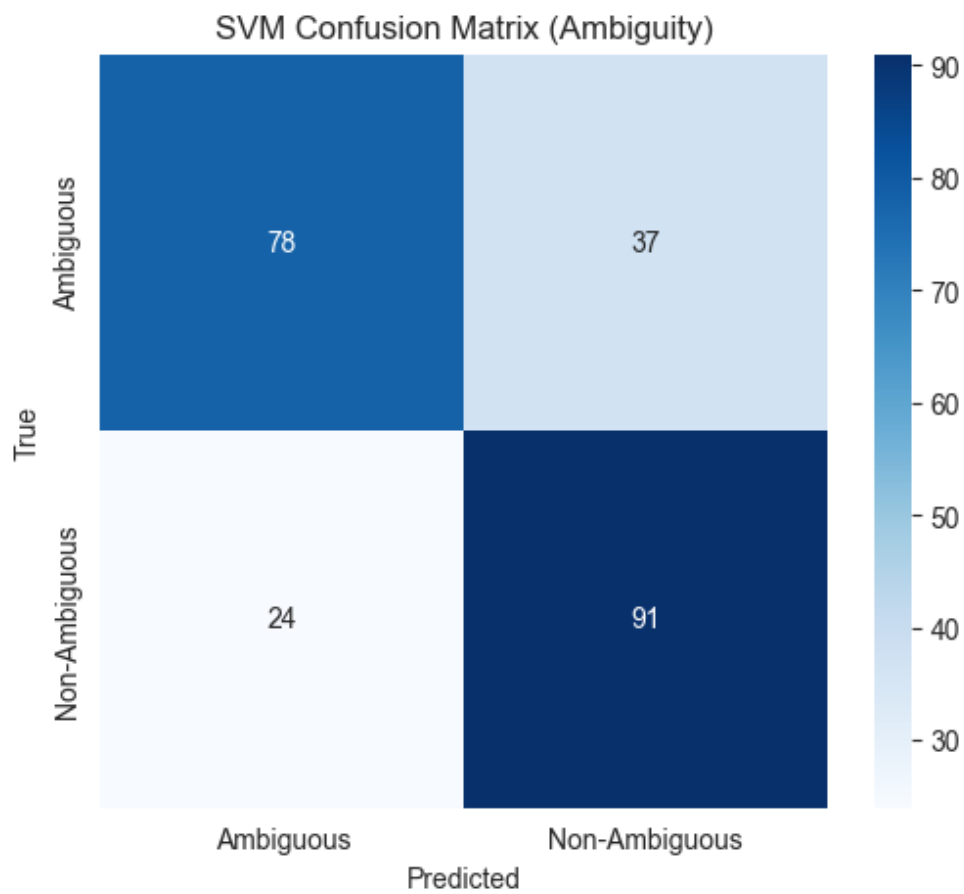


Fig. 22. Stage 3 LinearSVC confusion matrix for ambiguity classification

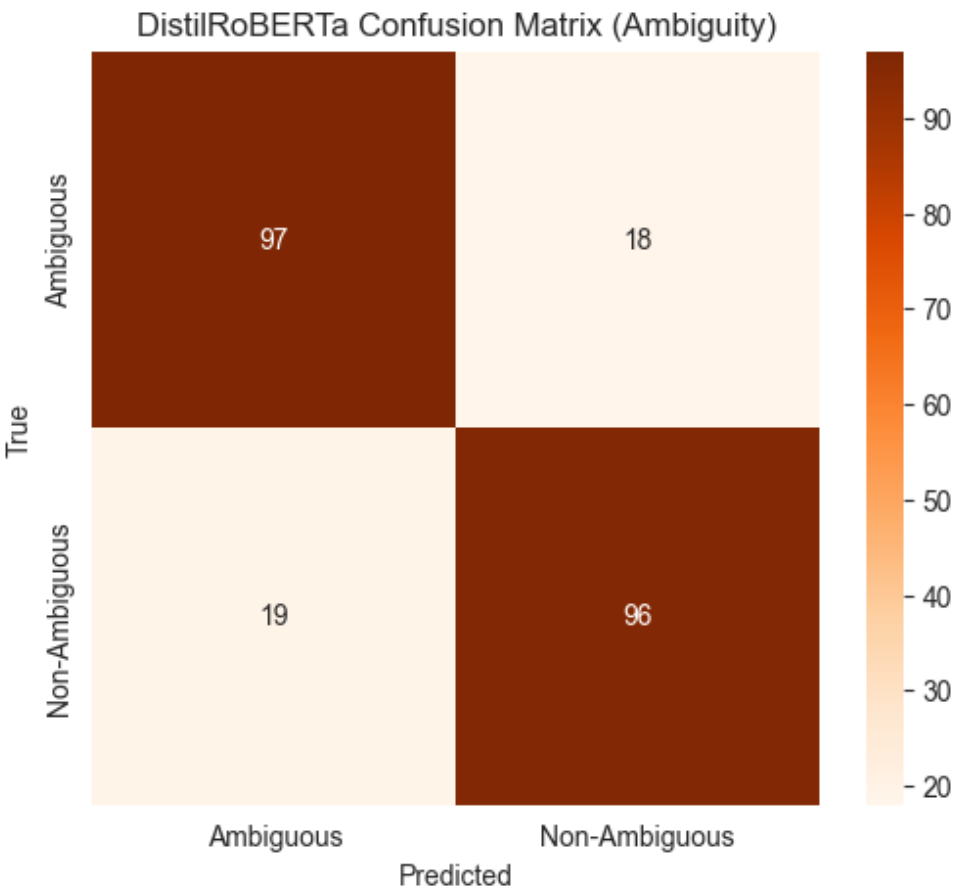


Fig. 23. Stage 3 DistilRoBERTa confusion matrix for ambiguity classification